Fundamentals of Database Systems

for Data Scientist

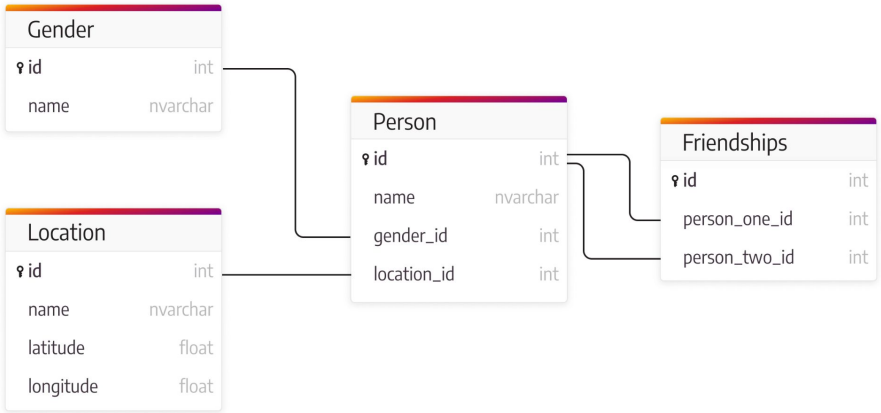
2025

Lê Thiện Hòa

Nội dung

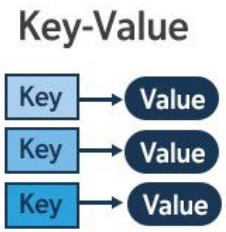
- SQL vs. No-SQL, Row-wise & Column-wise DB
- OLTP vs. OLAP
- ACID & BASE principles
- Data Warehouse, Data Mart
- Dimensional Modeling, Facts, Dimensions, Slowly Changing Dimensions (SCD)
- Normalization vs. Denormalization, Star Schema, Snowflake Schema
- Extract - Transform - Load (ETL), ELT
- Data Lake, Lakehouse
- Distributed Databases (HBase, Cassandra), CAP theorem

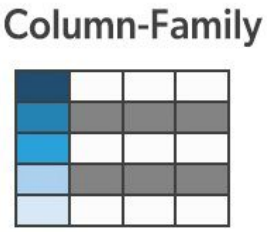
SQL databases

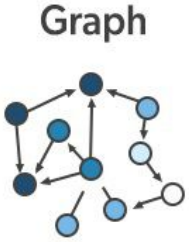


- **Có cấu trúc (Structured)**: dữ liệu lưu trong bảng (table), hàng (row), cột (column).
- **Ngôn ngữ chuẩn (SQL)**: dùng SQL để query, join, filter, aggregate.
- **Quan hệ (Relational)**: hỗ trợ liên kết giữa nhiều bảng bằng khóa chính/ngoại.
- **Tuân thủ ACID**: đảm bảo tính toàn vẹn dữ liệu trong giao dịch.
- **Phù hợp OLTP**: xử lý giao dịch nhanh, nhiều truy vấn nhỏ.
- **Schema cố định**: dữ liệu phải tuân theo thiết kế trước (schema-first).
- **Ví dụ**: MySQL, PostgreSQL, Oracle, SQL Server.

No-SQL databases







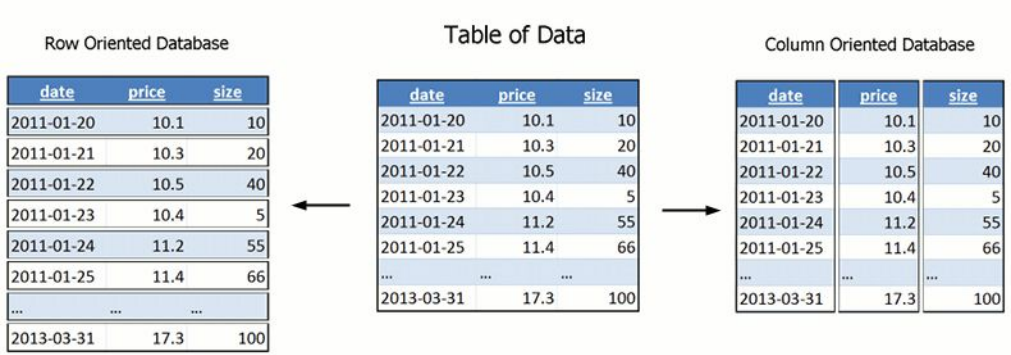


- **Linh hoạt schema**: không cần định nghĩa cố định trước (schema-less hoặc schema-flexible).
- **Đa dạng mô hình dữ liệu**: document, key-value, column-family, graph.
- **Khả năng scale-out**: phân tán theo chiều ngang, dễ mở rộng với dữ liệu lớn.
- **Tối ưu cho dữ liệu phi cấu trúc / bán cấu trúc** (JSON, log, sensor, mạng xã hội).
- **Hiệu năng cao** cho workload đặc thù (write-intensive, real-time query).
- **Không phải lúc nào cũng ACID**, thường theo **BASE**
- **Ví dụ**: MongoDB (document), Cassandra (column), Redis (key-value), Neo4j (graph).

When to choose SQL / No-SQL ?

Tiêu chí	Chọn SQL nếu	Chọn NoSQL nếu
Cấu trúc dữ liệu	Cần dữ liệu có cấu trúc, dạng bảng, schema cố định	Cần schema linh hoạt hoặc lưu dữ liệu phi cấu trúc/bán cấu trúc
Khả năng mở rộng	Scale dọc (tăng tài nguyên cho 1 server) là đủ	Cần scale ngang (thêm nhiều server) để xử lý traffic lớn
Xử lý giao dịch	Yêu cầu tuân thủ **ACID** (vd: giao dịch tài chính)	Chấp nhận **eventual consistency**, ưu tiên tính sẵn sàng cao
Nhu cầu truy vấn	Cần query SQL phức tạp: JOIN, aggregate	Chủ yếu lookup key-value hoặc query phân tán đơn giản
Yêu cầu hiệu năng	Ưu tiên tính toàn vẹn, chính xác hơn tốc độ	Ưu tiên tốc độ, xử lý real-time
Use case thực tế	Ngân hàng, ERP, CRM, ứng dụng doanh nghiệp truyền thống	Big data, IoT, mạng xã hội, CMS, phân tích real-time

Row-wise vs. Column-wise Databases



Loại DB	Row-wise	Column-wise
SQL	MySQL, PostgreSQL	Redshift, BigQuery, ClickHouse
NoSQL	MongoDB (document), Neo4j (graph)	Cassandra, HBase

Row-wise (hàng):

- Lưu dữ liệu theo dòng → đọc/ghi cả record.
- Phù hợp **OLTP**: nhiều transaction nhỏ, insert/update nhanh.

Column-wise (cột):

- Lưu dữ liệu theo cột → đọc/ghi tập trung theo từng thuộc tính.
- Phù hợp **OLAP**: phân tích, aggregate trên cột, truy vấn khối lượng lớn.
- Nén dữ liệu tốt, tăng tốc query analytic.

OLTP (Online Transaction Processing) vs. OLAP (Online Analytical Processing)

So sánh OLTP vs OLAP

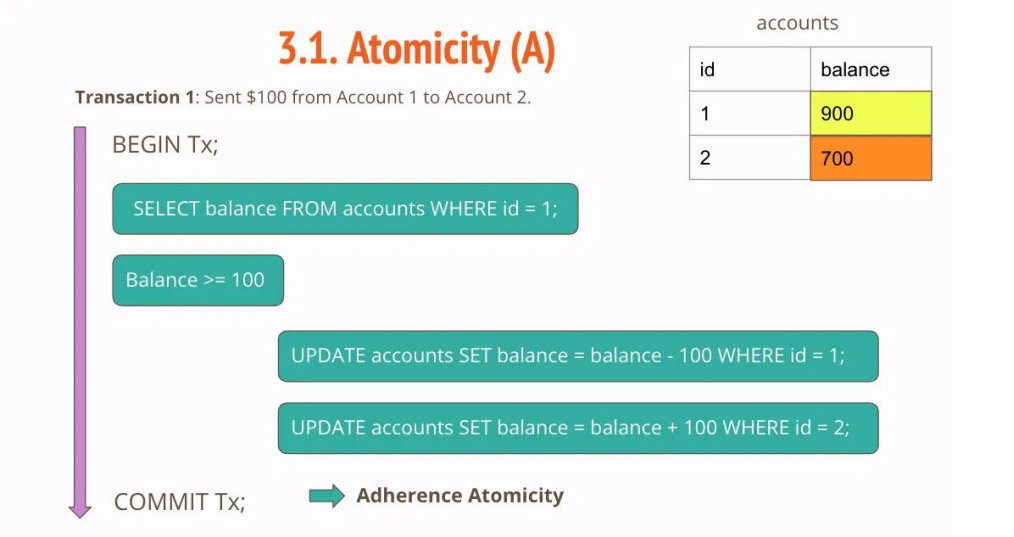
Tiêu chí	OLTP (Transaction)	OLAP (Analytics)
Mục tiêu	Xử lý giao dịch nhanh, chính xác	Phân tích, tổng hợp dữ liệu nhiều chiều
Dữ liệu	Chi tiết, thời gian thực	Đã tổng hợp, lịch sử, nhiều nguồn
Truy vấn	Ngắn, đơn giản (insert/update)	Dài, phức tạp (aggregate, join lớn)
Tối ưu	**Ghi nhanh, tính toàn vẹn (ACID)**	**Đọc nhanh, phân tích khối lượng lớn**
Quy mô	Hàng ngàn giao dịch/giây	Hàng triệu bản ghi để phân tích
Câu hỏi ví dụ	*Ai đã mua sản phẩm X?*	*Có bao nhiêu người đã mua sản phẩm X?*
Use case	Ngân hàng, e-commerce checkout, ERP, CRM	Data warehouse, BI dashboard, báo cáo tài chính
Ví dụ DB	MySQL, PostgreSQL, Oracle, SQL Server	Amazon Redshift, Snowflake, Google BigQuery, ClickHouse

ACID principles

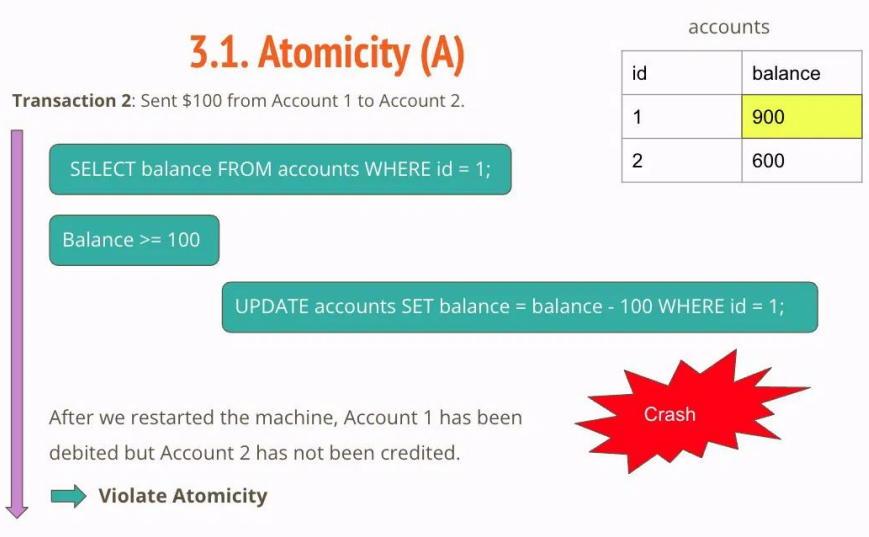
Trong hệ thống **OLTP**, dữ liệu giao dịch cần được **đảm bảo đúng, an toàn và đáng tin cậy**.
Bộ nguyên tắc **ACID** là tiêu chuẩn để **đảm bảo chất lượng** này trong mọi cơ sở dữ liệu quan hệ.

- **A – Atomicity (Tính nguyên tử)**
  - Giao dịch là tất cả hoặc không gì cả (all-or-nothing).
  - VD: Chuyển tiền, nếu debit thành công mà credit lỗi → rollback toàn bộ.
- **C – Consistency (Tính nhất quán)**
  - Sau mỗi giao dịch, dữ liệu luôn ở trạng thái hợp lệ, đúng quy tắc.
  - VD: Tổng tiền trong hệ thống không được thay đổi bất hợp lý.
- **I – Isolation (Tính cô lập)**
  - Các giao dịch chạy song song nhưng không ảnh hưởng đến nhau.
  - VD: 2 người cùng đặt 1 vé, chỉ 1 người thành công.
- **D – Durability (Tính bền vững)**
  - Khi giao dịch commit, dữ liệu được lưu vĩnh viễn, không mất dù có sự cố.
  - VD: Sau khi báo “chuyển tiền thành công”, dữ liệu vẫn còn dù server tắt.

ACID principles - Atomicity



Trường hợp tuân thủ tính nguyên tử



Trường hợp ***không*** tuân thủ tính nguyên tử

ACID principles - Consistency

- Đảm bảo dữ liệu luôn hợp lệ sau mỗi transaction.
- Transaction không làm vi phạm constraints/rules của database.
- Dựa trên các cơ chế:
  - Constraints: **PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, NOT NULL**.
  - Schema rules: định nghĩa kiểu dữ liệu, quan hệ giữa bảng.
  - Rollback: nếu vi phạm thì DB tự hoàn tác.

- Ví dụ:
  - Tài khoản không thể có số dư âm (CHECK balance >= 0).
  - Không thể chèn đơn hàng nếu customer_id không tồn tại (FOREIGN KEY).

ACID principles - Consistency

Giả sử ta có bảng:

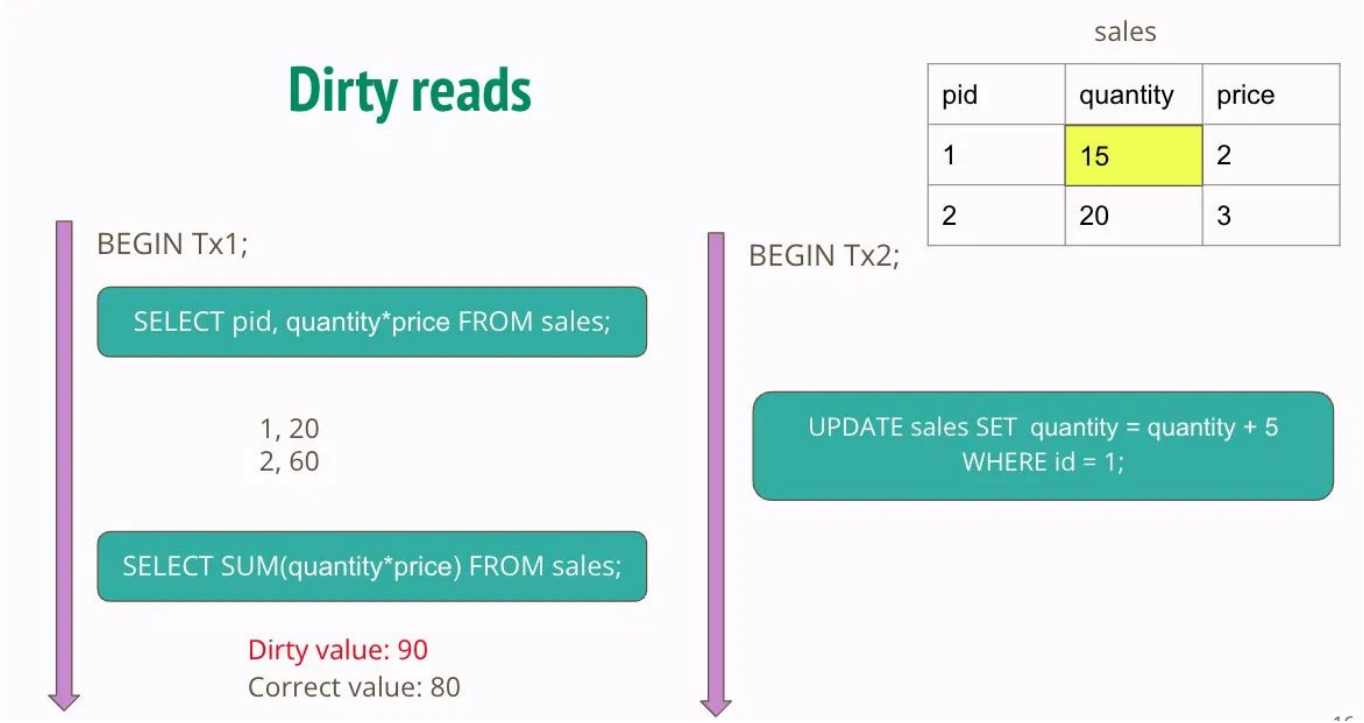
```sql
CREATE TABLE accounts (
    account_id INT PRIMARY KEY,
    balance DECIMAL CHECK (balance >= 0),
    customer_id INT,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

- Khi chạy:

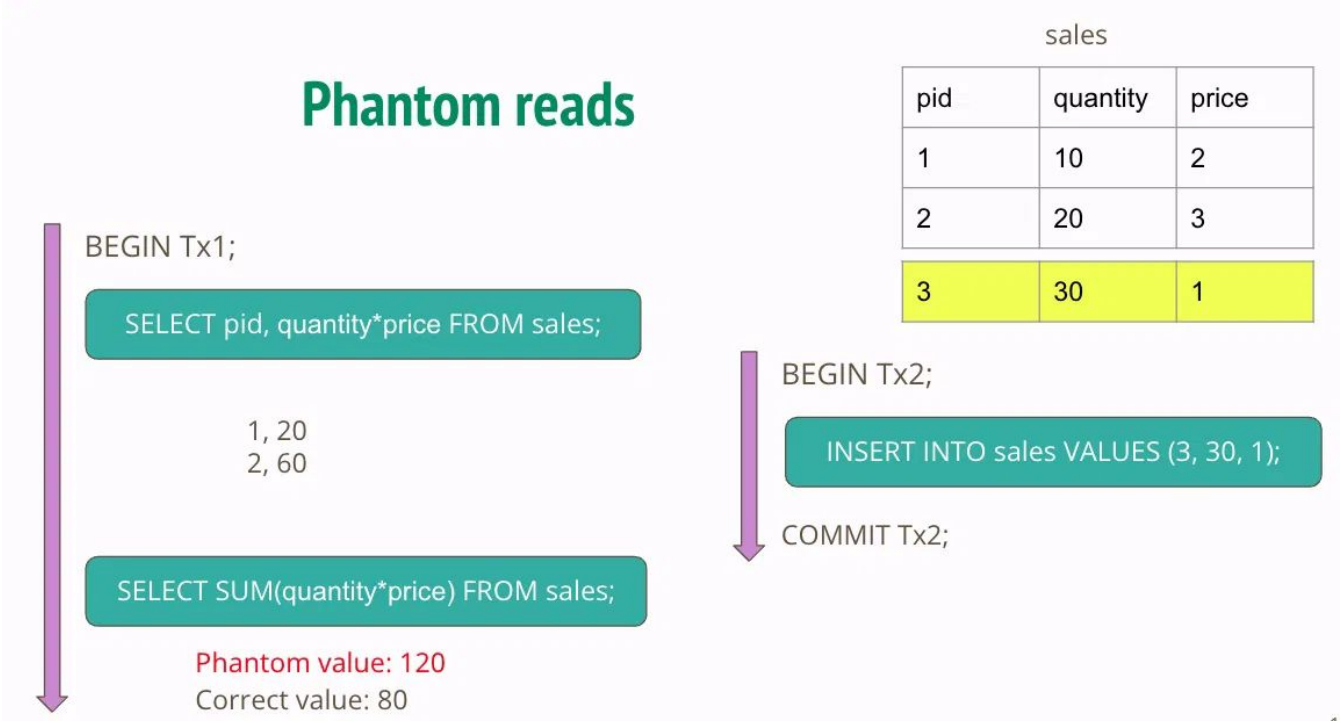
```sql
INSERT INTO accounts VALUES (1, -500, 10);
```

→ Engine check `CHECK (balance >= 0)` → thấy -500 không hợp lệ → rollback.

ACID principles - Isolation



ACID principles - Isolation



ACID principles - Durability

- Khi transaction đã commit, dữ liệu sẽ không bao giờ mất, kể cả khi hệ thống crash.
- Đảm bảo bởi:
  - **Write-Ahead Logging** (WAL): ghi log ra disk trước khi báo commit thành công.
  - **Flush ra storage**: dữ liệu chỉ báo thành công sau khi ghi an toàn xuống disk.
  - **Crash recovery**: khi restart, DB đọc lại log để khôi phục.

- Ví dụ:
  - Bạn chuyển 100k từ A sang B → sau commit thành công, dù mất điện → khi bật lại, số dư vẫn được cập nhật chính xác.

BASE principles (cho No-SQL / OLAP)

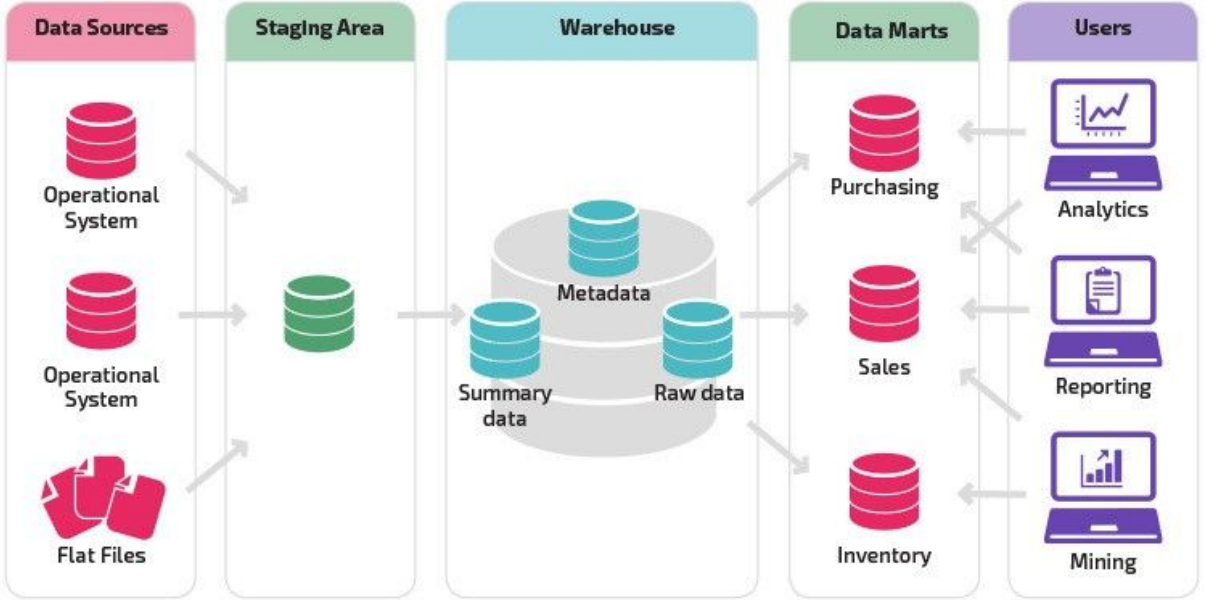
BASE là cách tiếp cận ngược lại, ưu tiên khả năng mở rộng (scalability) và hiệu năng hơn là tính chặt chẽ.

- **B – Basically Available**: hệ thống luôn sẵn sàng để đọc/ghi (có thể trả dữ liệu tạm thời).
- **S – Soft state**: trạng thái dữ liệu có thể thay đổi theo thời gian (ngay cả khi không có input mới), vì dữ liệu được replicate giữa nhiều node.
- **E – Eventual consistency**: dữ liệu có thể chưa đồng bộ ngay, nhưng cuối cùng sẽ đồng bộ trên tất cả các node.

ACID vs. BASE

Đặc tính	ACID	BASE
Viết tắt	Atomicity, Consistency, Isolation, Durability	Basically Available, Soft state, Eventual consistency
Tính sẵn sàng	Không cao bằng BASE (ưu tiên tính chính xác)	Rất cao, luôn sẵn sàng, ưu tiên availability
Tính toàn vẹn	Rất chặt chẽ, dữ liệu luôn hợp lệ và tuân thủ constraint	Lỏng hơn, có thể tạm thời không đồng bộ, nhưng cuối cùng sẽ nhất quán
Khả năng mở rộng	Khó scale ngang, thường scale dọc	Dễ scale ngang, hỗ trợ hệ thống phân tán rất lớn
Thích hợp cho	Tài chính, ngân hàng, ERP, nơi cần **chính xác tuyệt đối**	Mạng xã hội, e-commerce, big data, nơi **chấp nhận sai lệch tạm thời**
Ví dụ	Oracle, PostgreSQL, MySQL (InnoDB), SQL Server	Cassandra, DynamoDB, Couchbase, MongoDB (một số chế độ)

Data Warehouse Layers



- Hệ thống gốc: ERP, CRM, Core Banking, Web/App, log files...
- Nơi dữ liệu raw được load về.
- Dùng để làm sạch, chuẩn hóa, loại bỏ trùng lặp.
- Dữ liệu đã được tổ chức theo mô hình (**Star**/**Snowflake**).
- Gồm **Fact table** & **Dimension table**.
- Lưu giữ dữ liệu lịch sử, sẵn sàng cho phân tích.
- Thu gọn dữ liệu theo nhu cầu của từng phòng ban (Sales, Finance, Marketing...).
- Tối ưu cho báo cáo nhanh.
- Dashboard , BI tools (Power BI, Tableau), hoặc API cho AI/ML.

Dimensional Modeling

Phương pháp tổ chức dữ liệu (trong Data Warehouse)

Facts

Measurement (ex. profit)

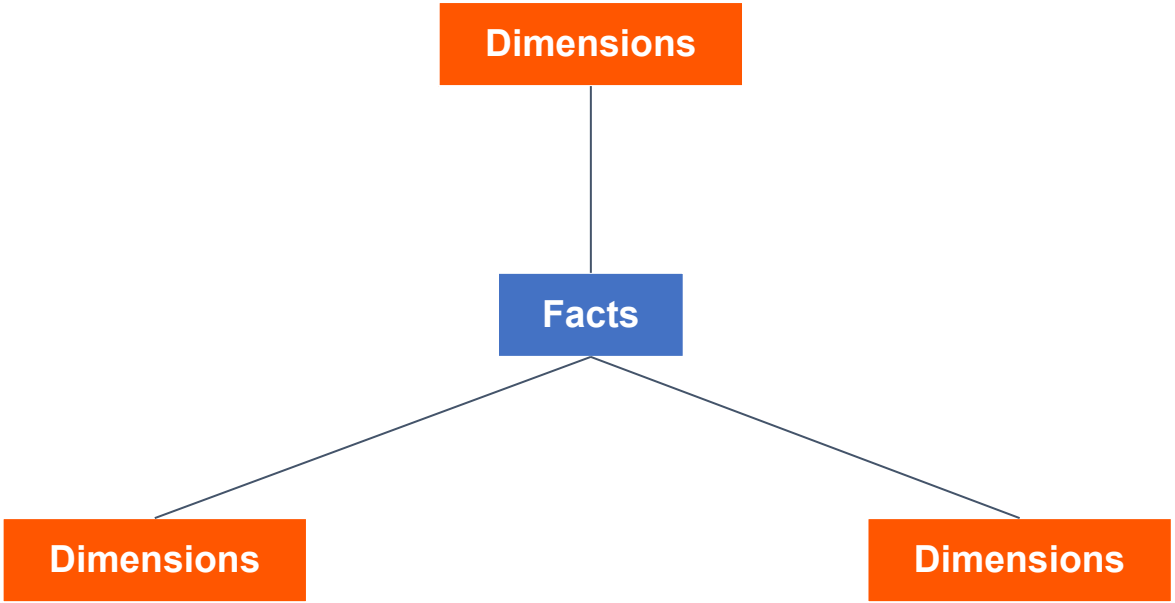
Dimensions

Context (ex. category, period)

Profit by category

Profit by year

Dimensional Modeling - Star schema



Star schema

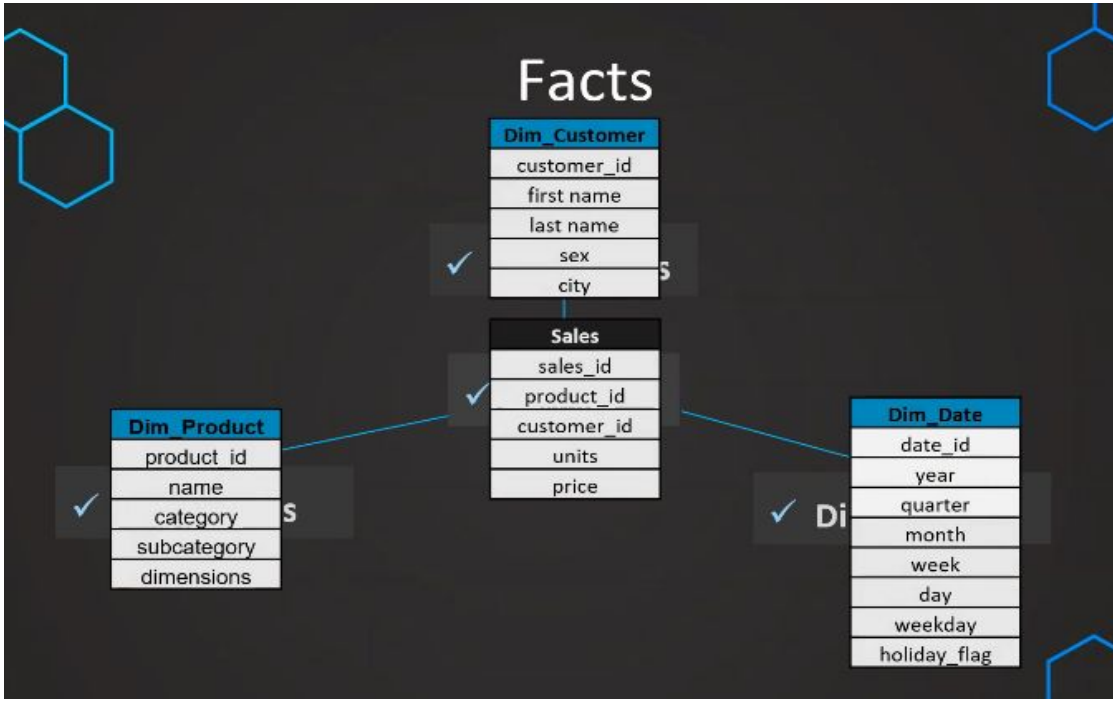
- Kỹ thuật ưu tiên để cấu trúc dữ liệu
- Thường được sử dụng trong DWH
- Tối ưu hóa để truy xuất dữ liệu nhanh hơn
- Hướng đến hiệu suất và khả năng sử dụng
- Báo cáo được thiết kế / OLAP

Dimensional Modeling

id	date	product	category	customer_id	profit
1	1/2/2022	Fulltoss Tangy Tomato	Vegetables	2	$23
2	1/2/2022	Chilli - Green, Organically Grown	Snacks	2	$12
3	1/2/2022	Masala Powder	Herbs	5	$93
4	1/2/2022	Cheese Cracker (Mcvities)	Snacks	1	$23
5	1/2/2022	Centre Filled Chocolate Cake	Snacks	5	$21



Facts



Bảng chứa **số liệu kinh doanh** (measures).

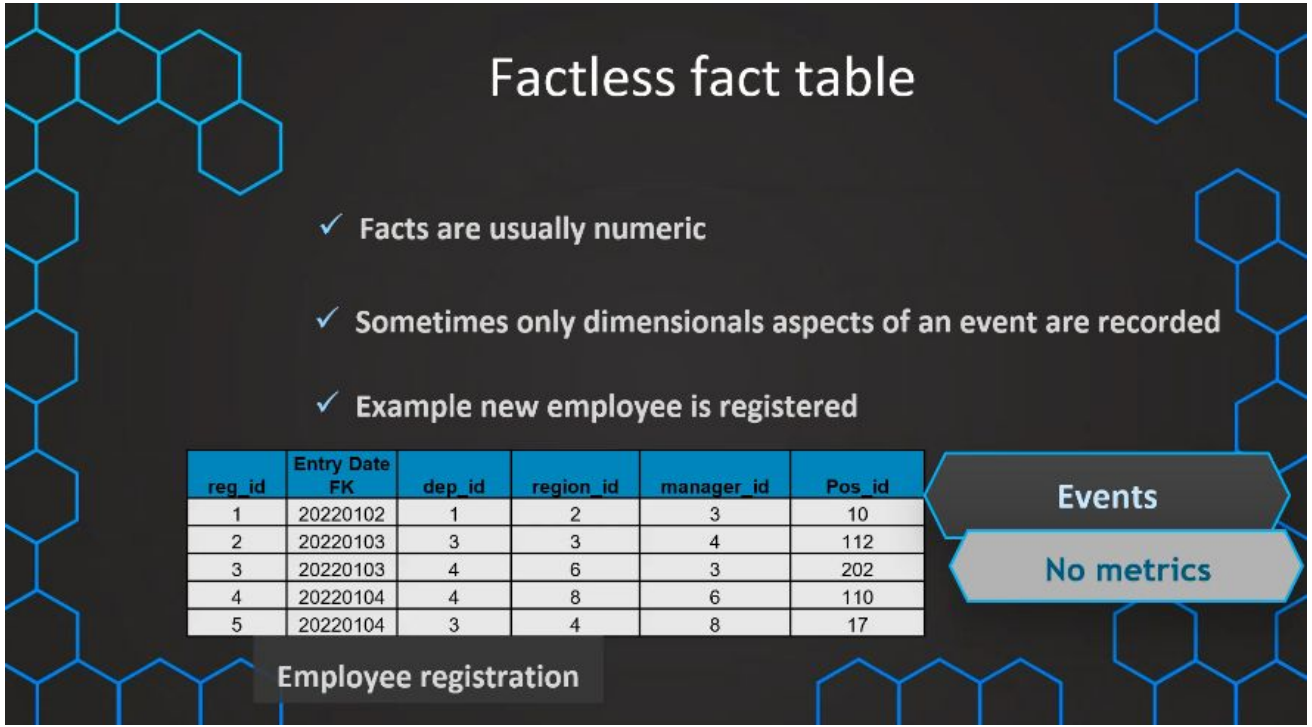
Đặc điểm:

- Rất nhiều dòng (hàng triệu).
- Fact Table: PK, FK, **Facts**.
- Có các measure: doanh thu, số lượng, chi phí...

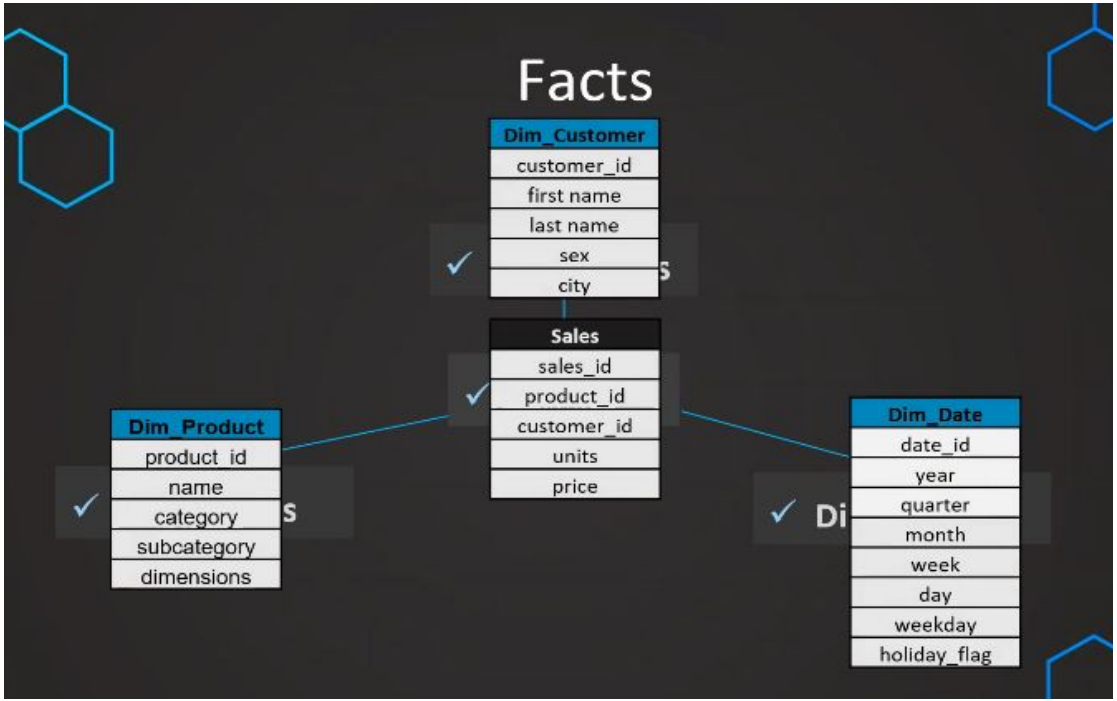
Vai trò:

- Trung tâm mô hình Dimensional.

Factless Fact Table



Dimensions



Bảng chứa **ngữ cảnh mô tả** (who, what, where, when).

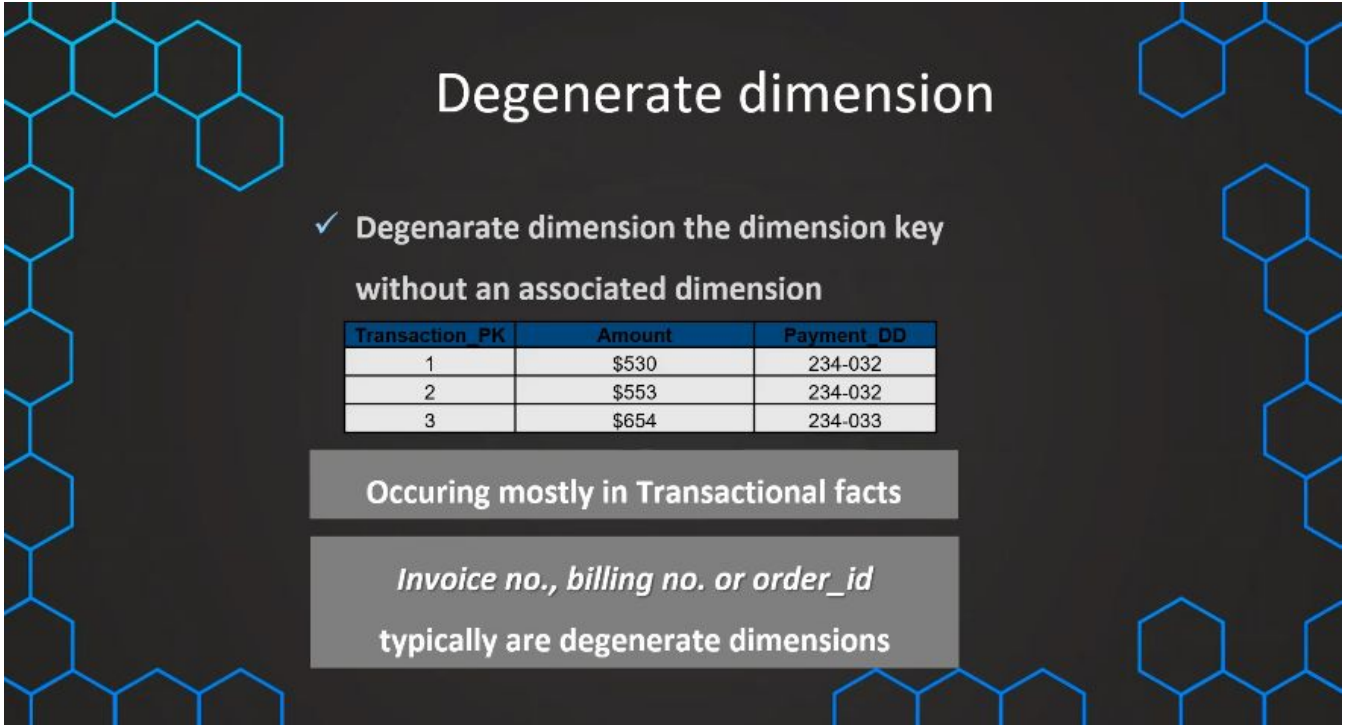
Đặc điểm:

- Ít dòng hơn Fact.
- **Nhiều cột mô tả** (tên, loại, khu vực...).
- Dimension Table: PK, **Dimension**, (FK)

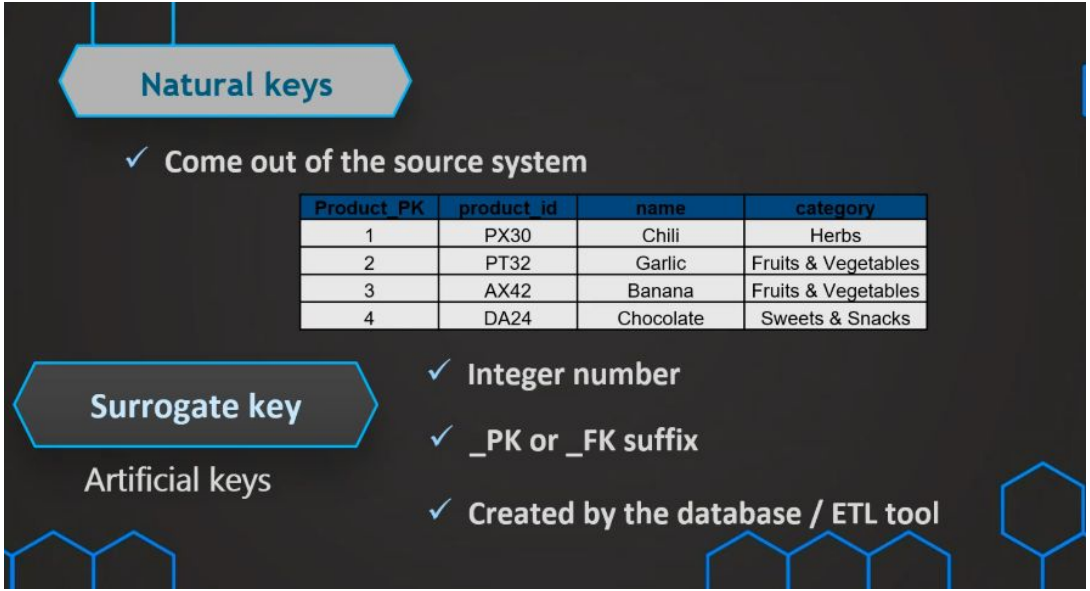
Vai trò:

- Giúp phân tích số liệu theo nhiều chiều: sản phẩm, khách hàng, thời gian…

Degenerate Dimensions



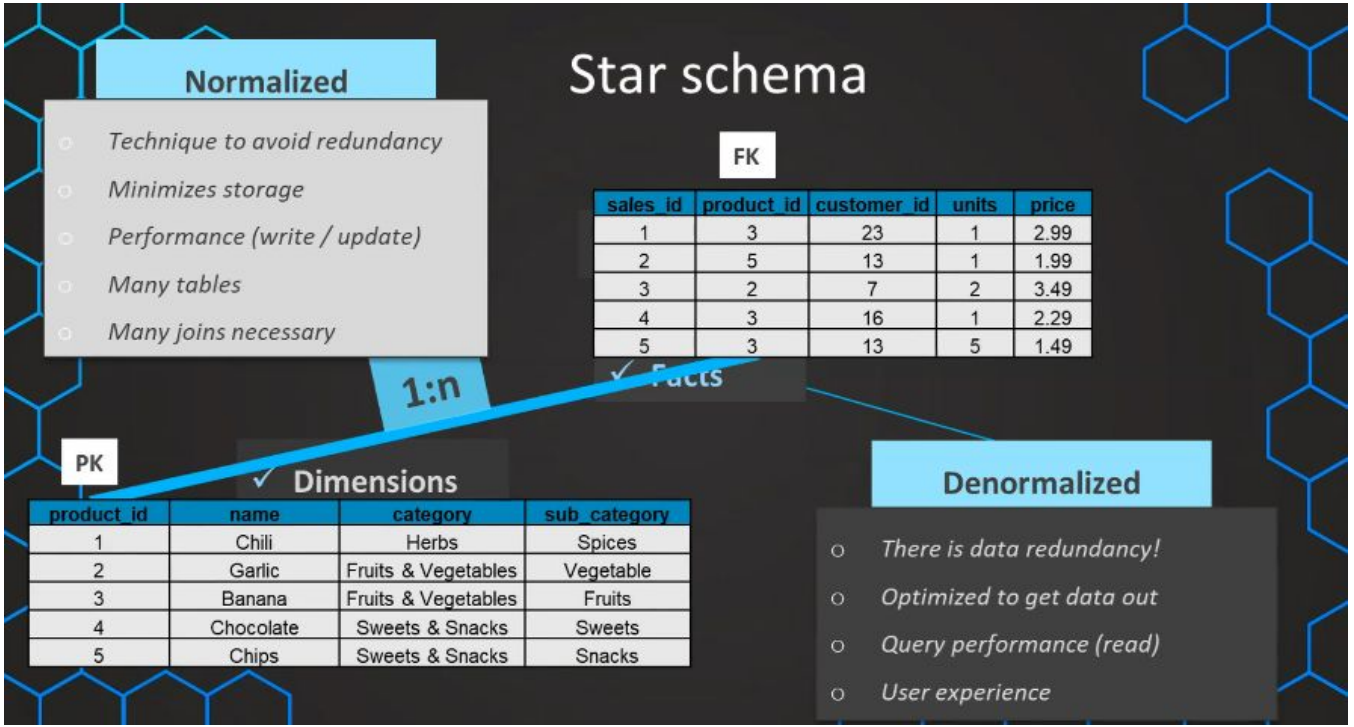
Surrogate Keys vs. Natural Key



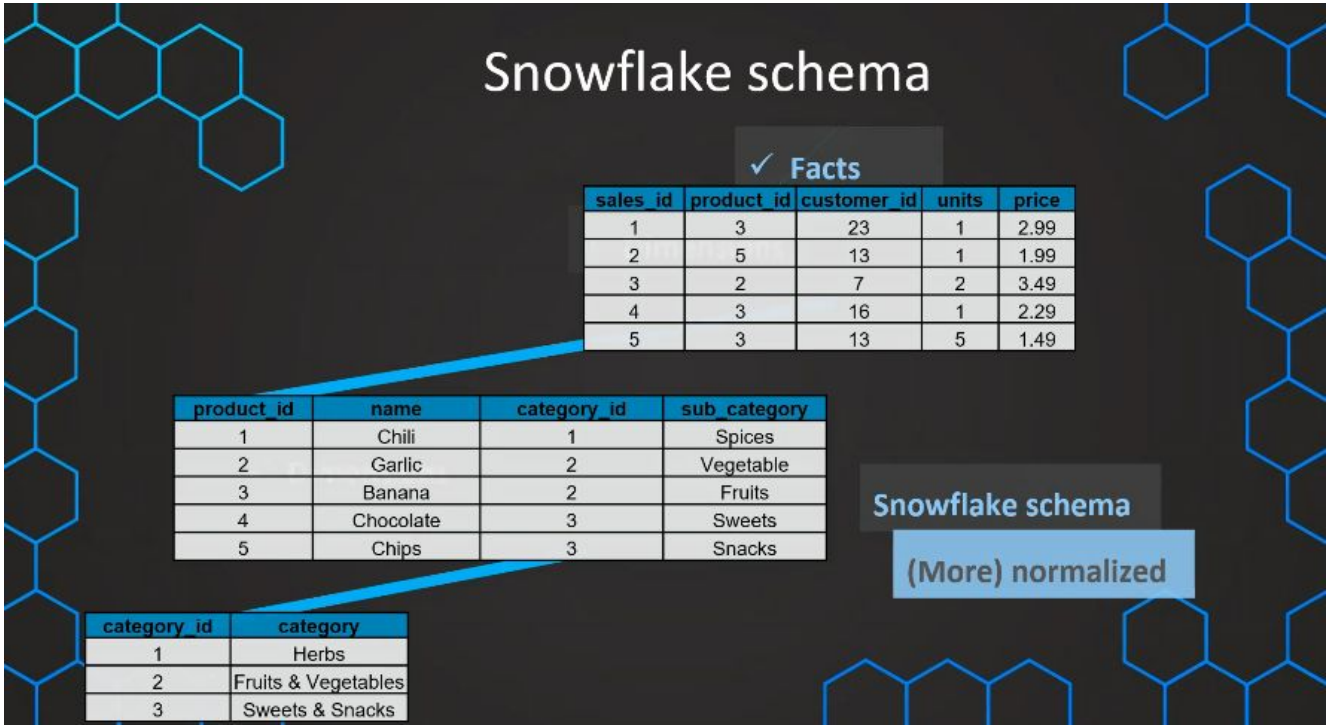
- Cải thiện hiệu suất (ít lưu trữ hơn/kết nối tốt hơn)
- Xử lý các giá trị giả (giá trị null, giá trị bị thiếu)
- Đôi khi, thậm chí không có khóa tự nhiên nào khả dụng

=> Luôn sử dụng Khóa Thay thế trong các bảng làm Khóa Chính (PK) hoặc Khóa Chính (FK)

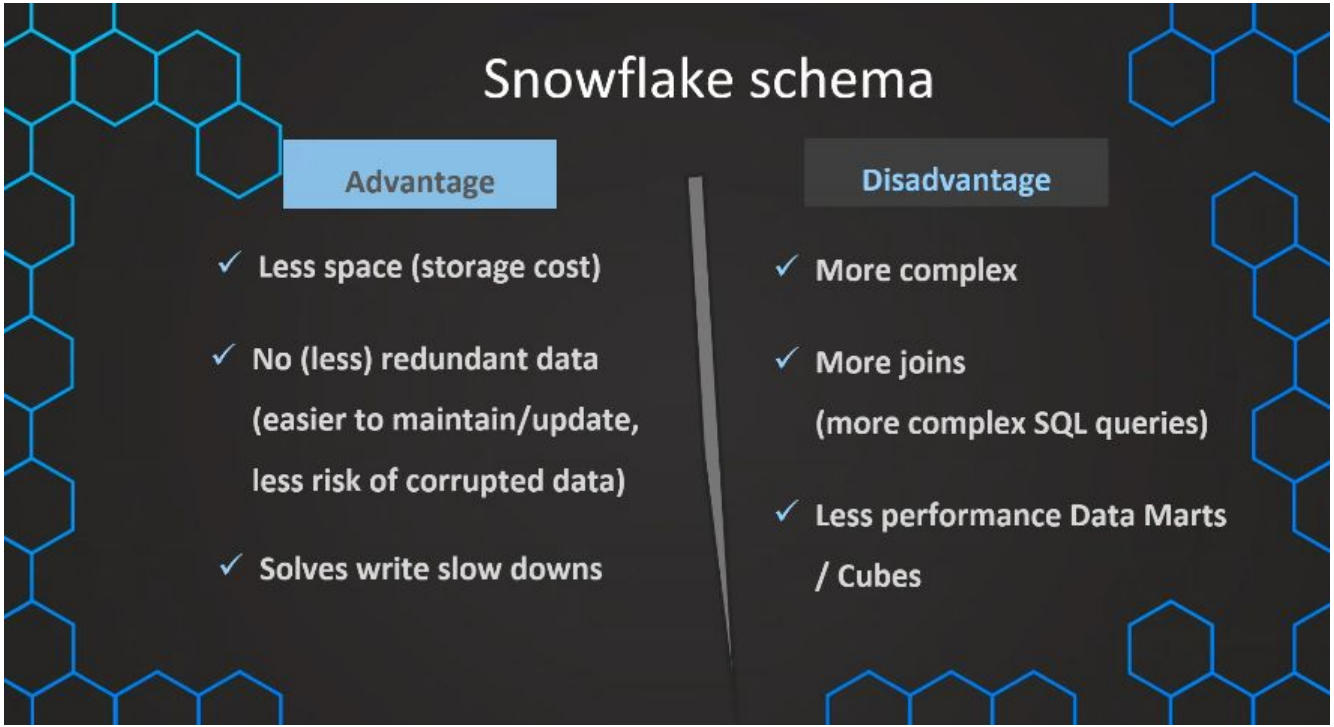
Normalization vs. Denormalization



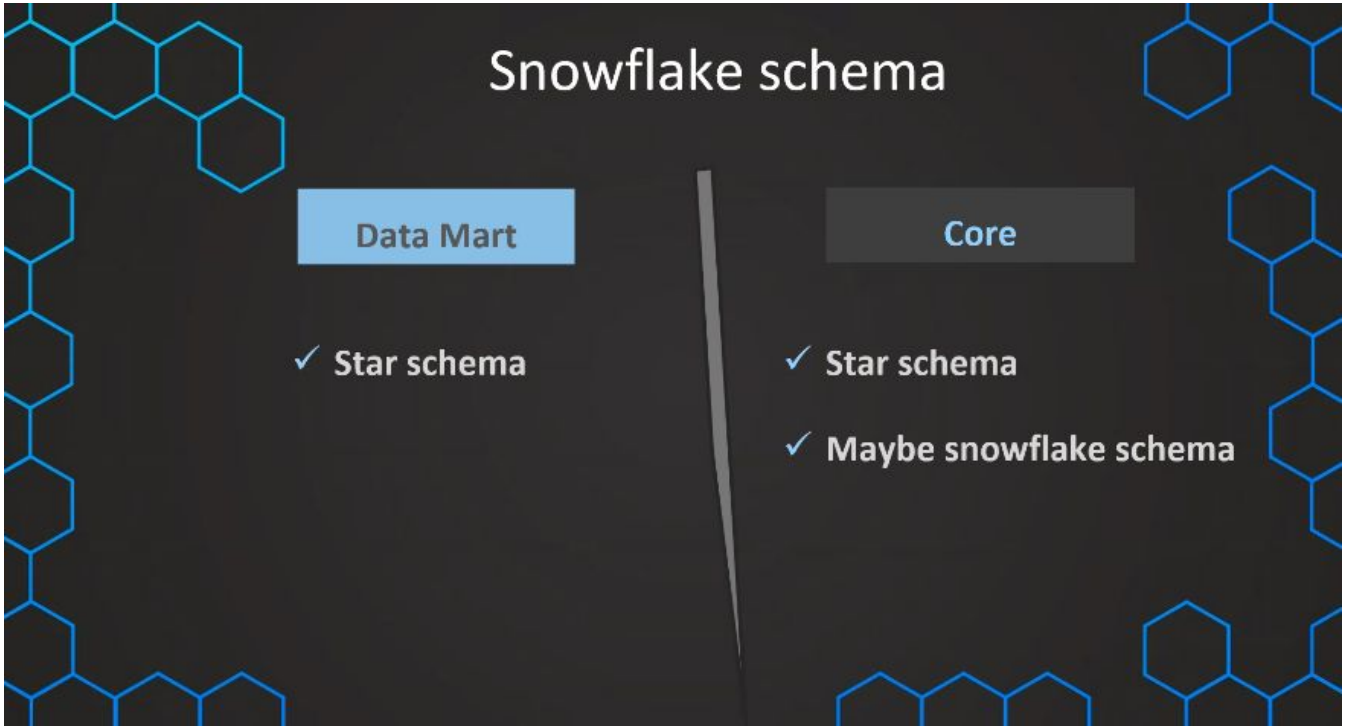
Snowflake Schema



Snowflake Schema



Star Schema vs. Snowflake Schema

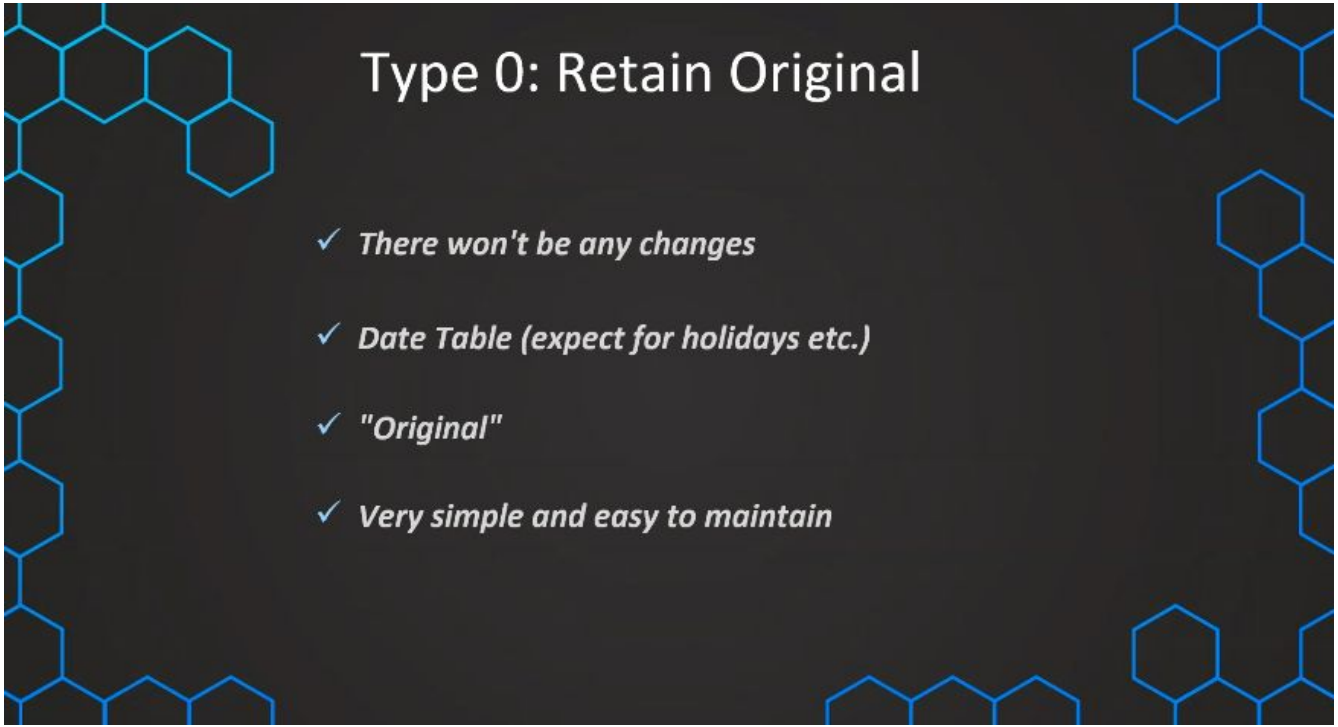


Slowly Changing Dimensions (SCD)

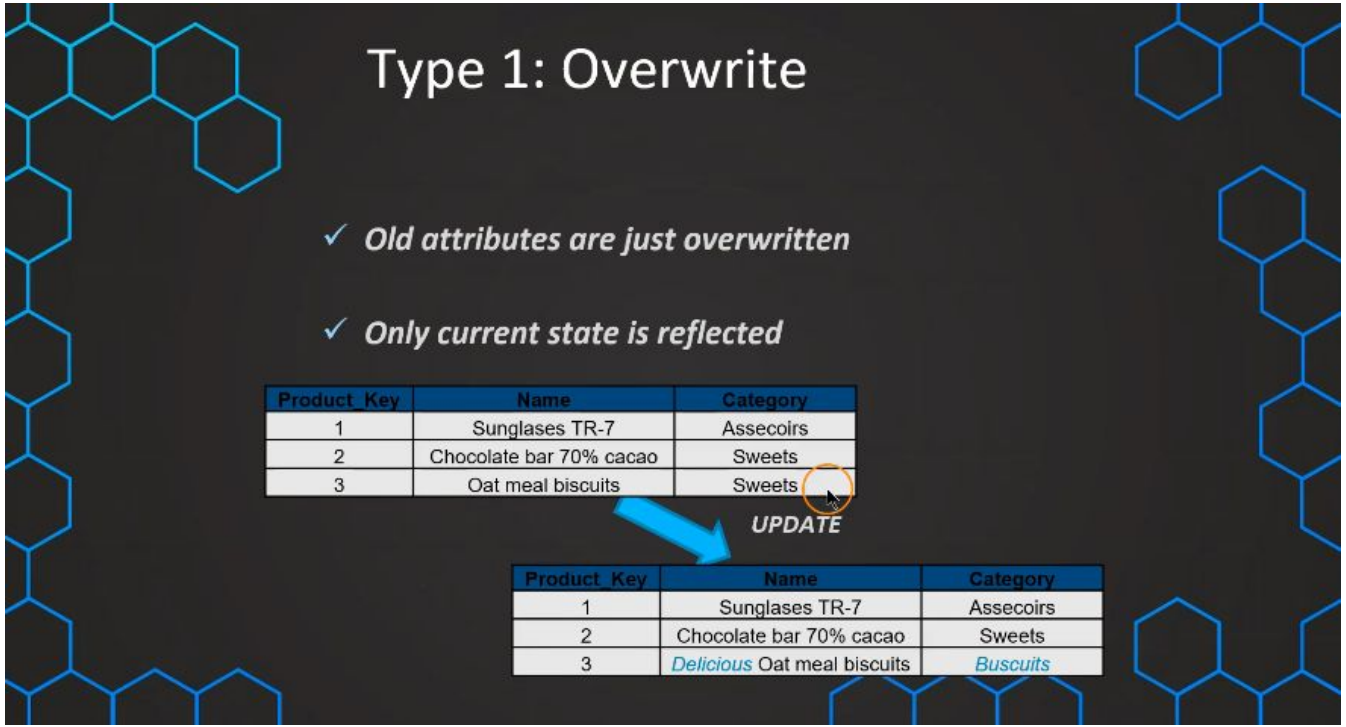
- Cho đến nay chúng ta vẫn giả định là Dimension không bao giờ thay đổi
- Thực ra chúng thường khá tĩnh…
- Nhưng đôi khi, chúng thực sự thay đổi trong thế giới thực…

- Cần một chiến lược để xử lý khi có những thay đổi về Dimension => **Kimball (1995)**

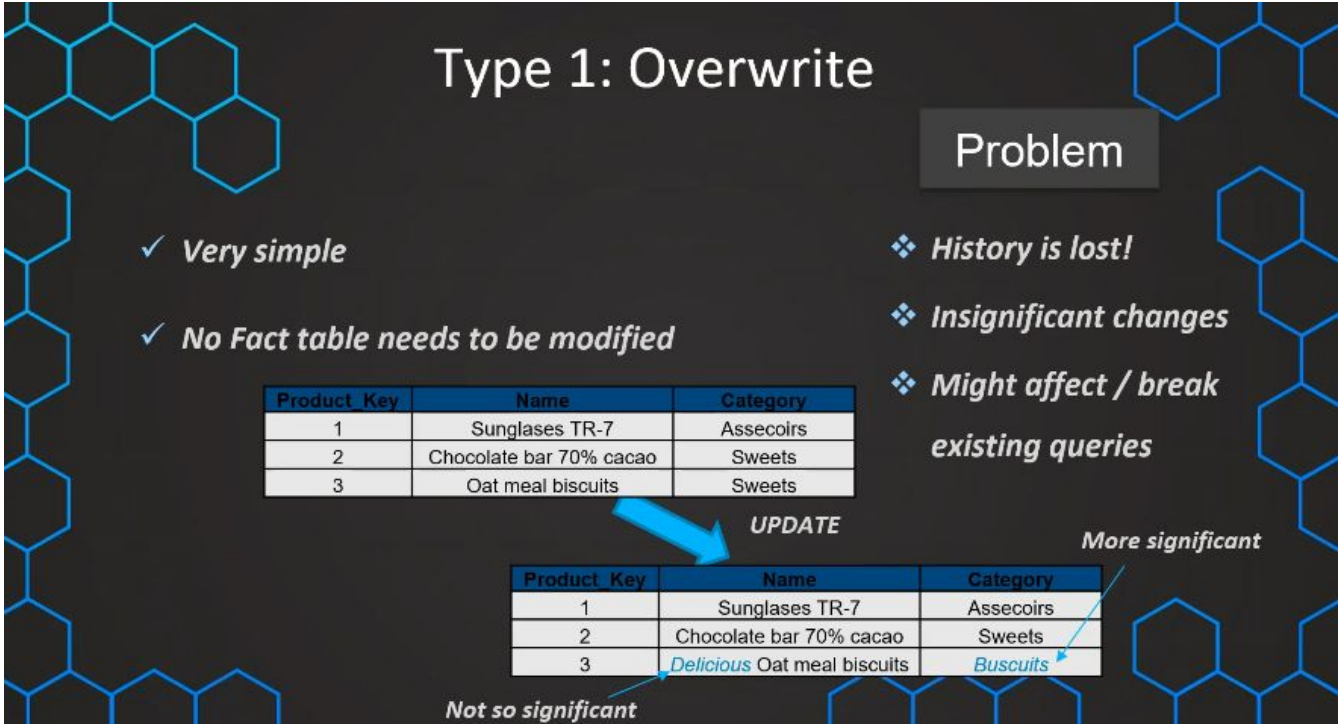
Slowly Changing Dimensions (SCD) - Type 0 (Original)



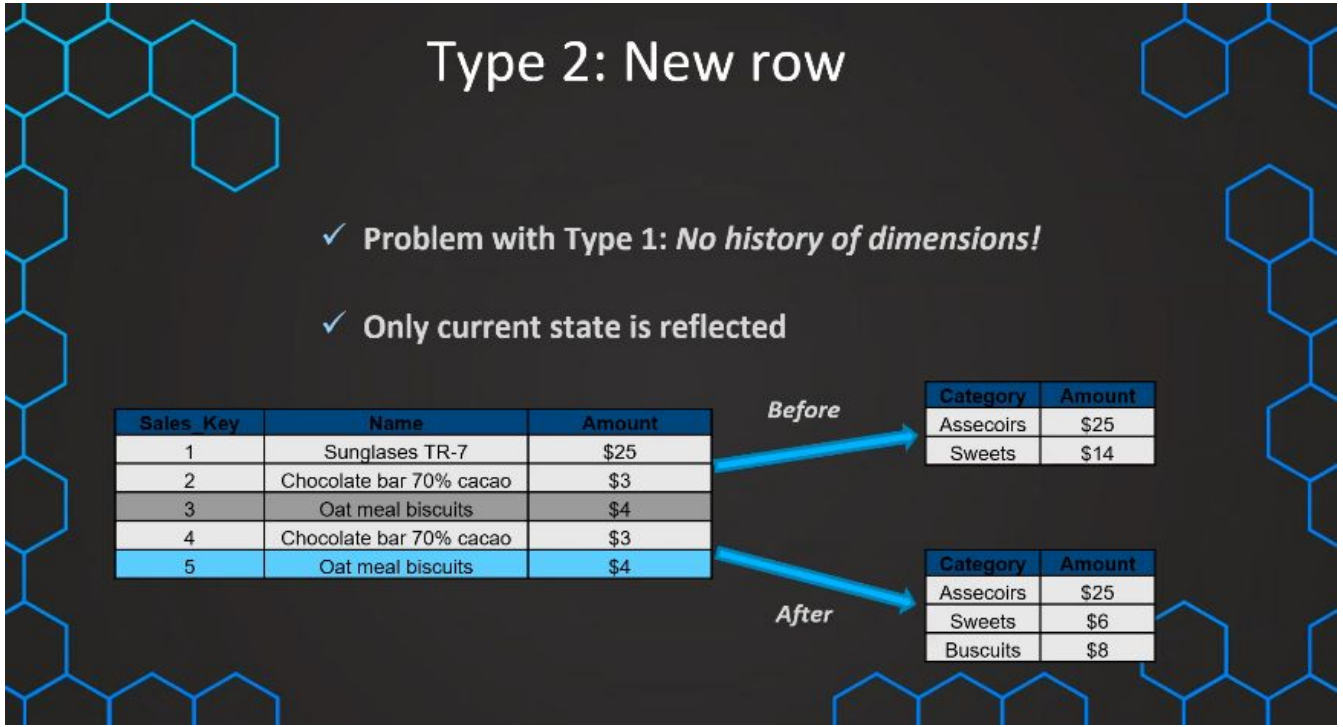
Slowly Changing Dimensions (SCD) - Type 1 (Overwrite)



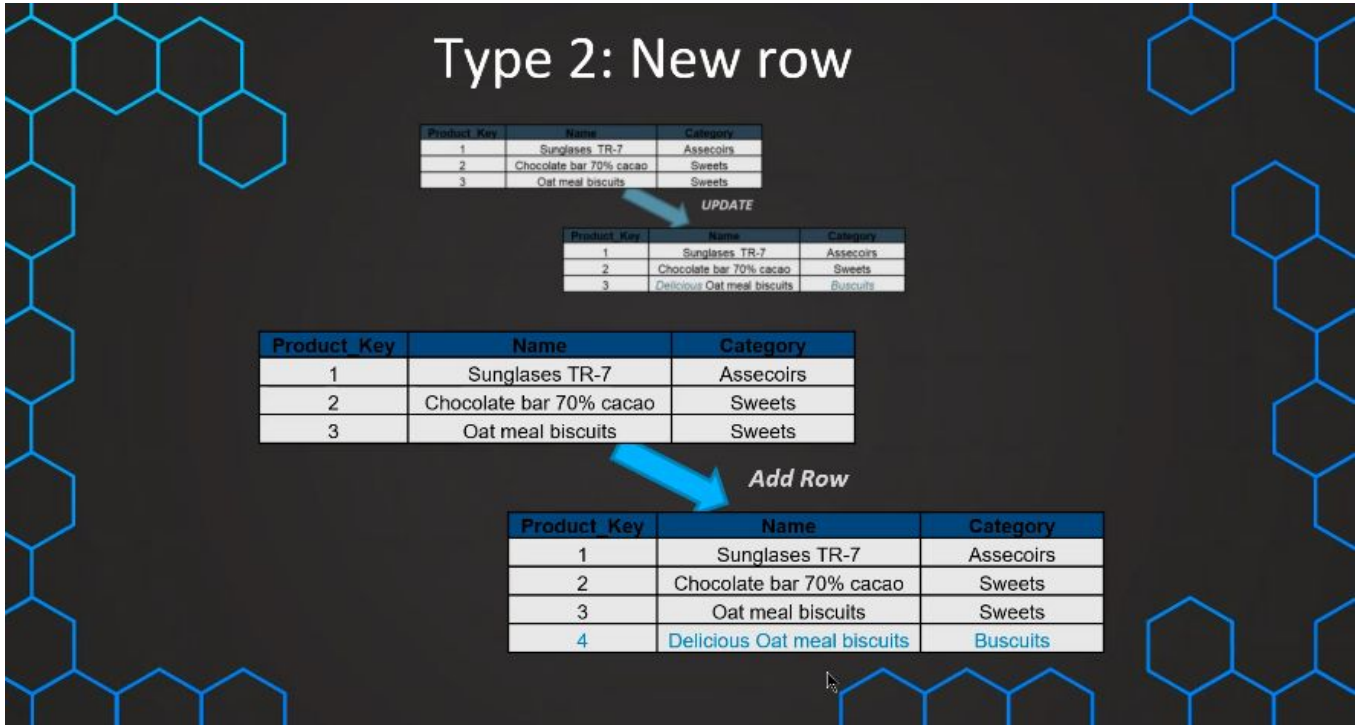
Slowly Changing Dimensions (SCD) - Type 1 (Overwrite)



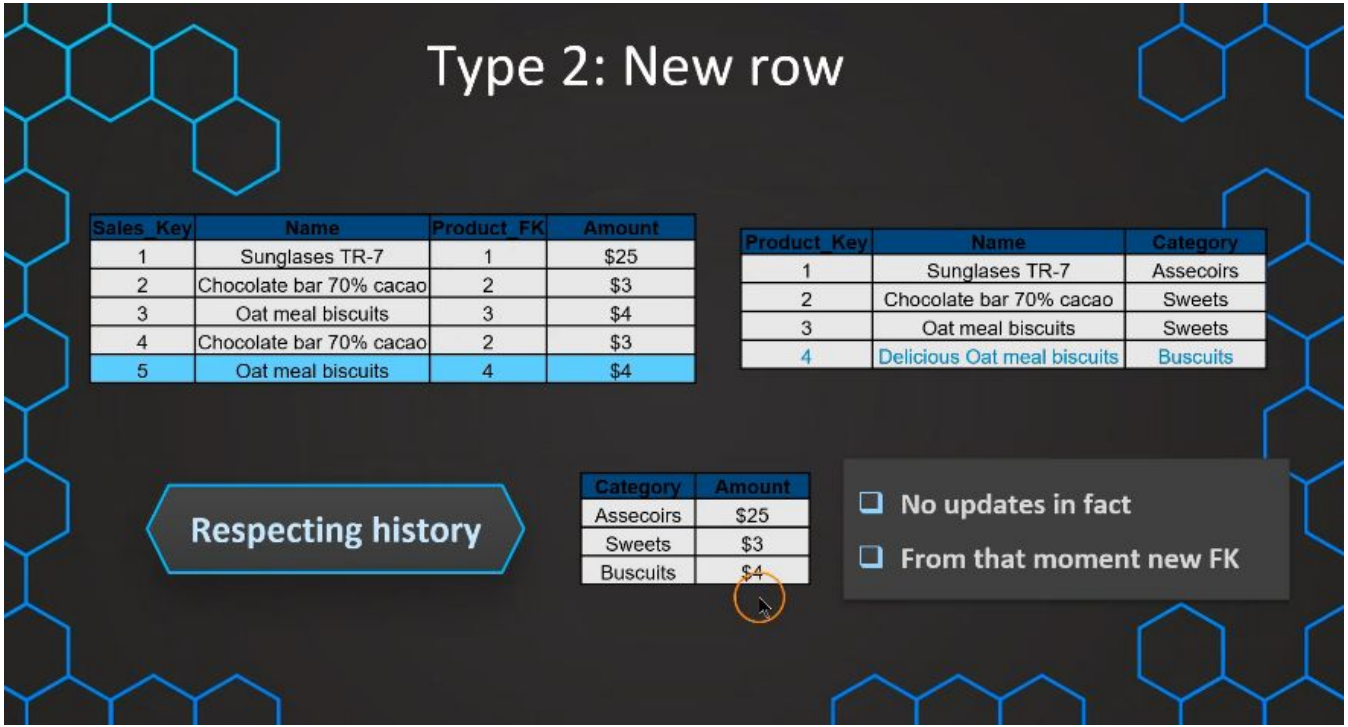
Slowly Changing Dimensions (SCD) - Type 2 (Additional Row)



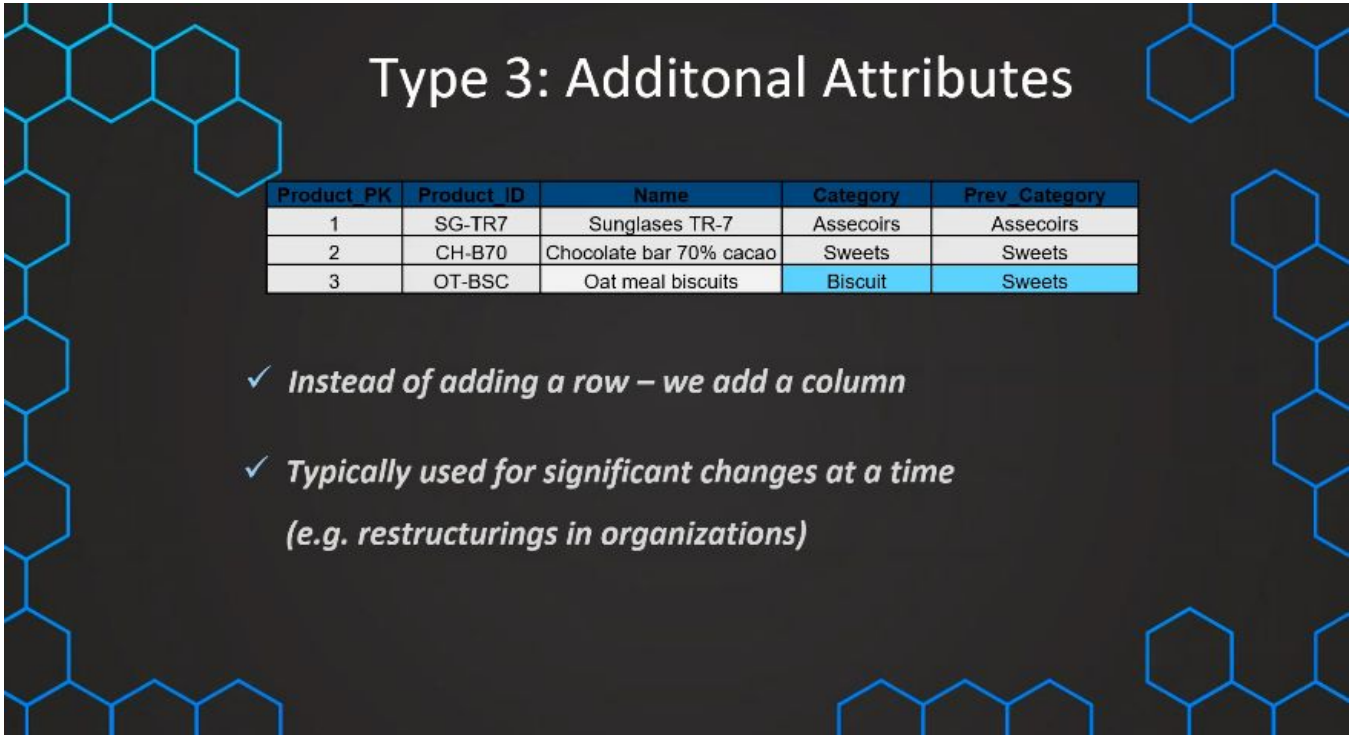
Slowly Changing Dimensions (SCD) - Type 2 (Additional Row)



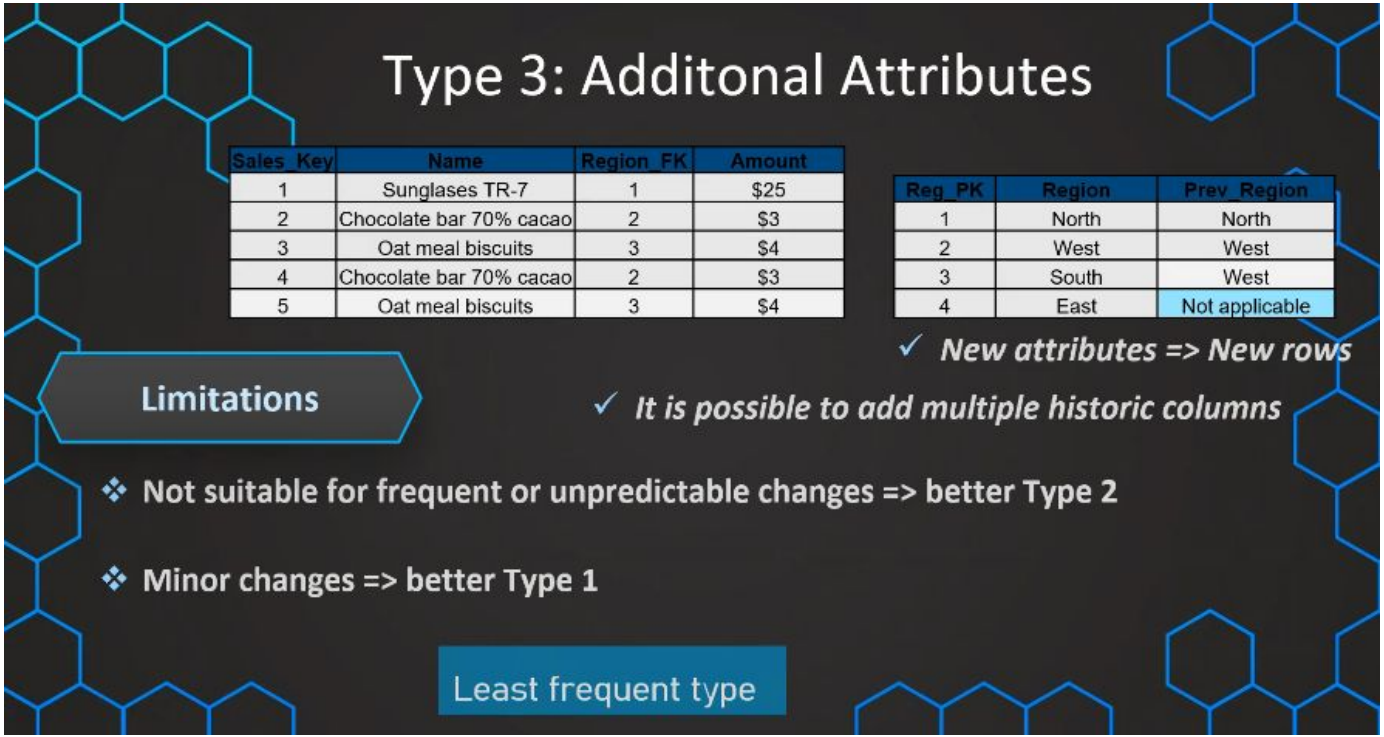
Slowly Changing Dimensions (SCD) - Type 2 (Additional Row)



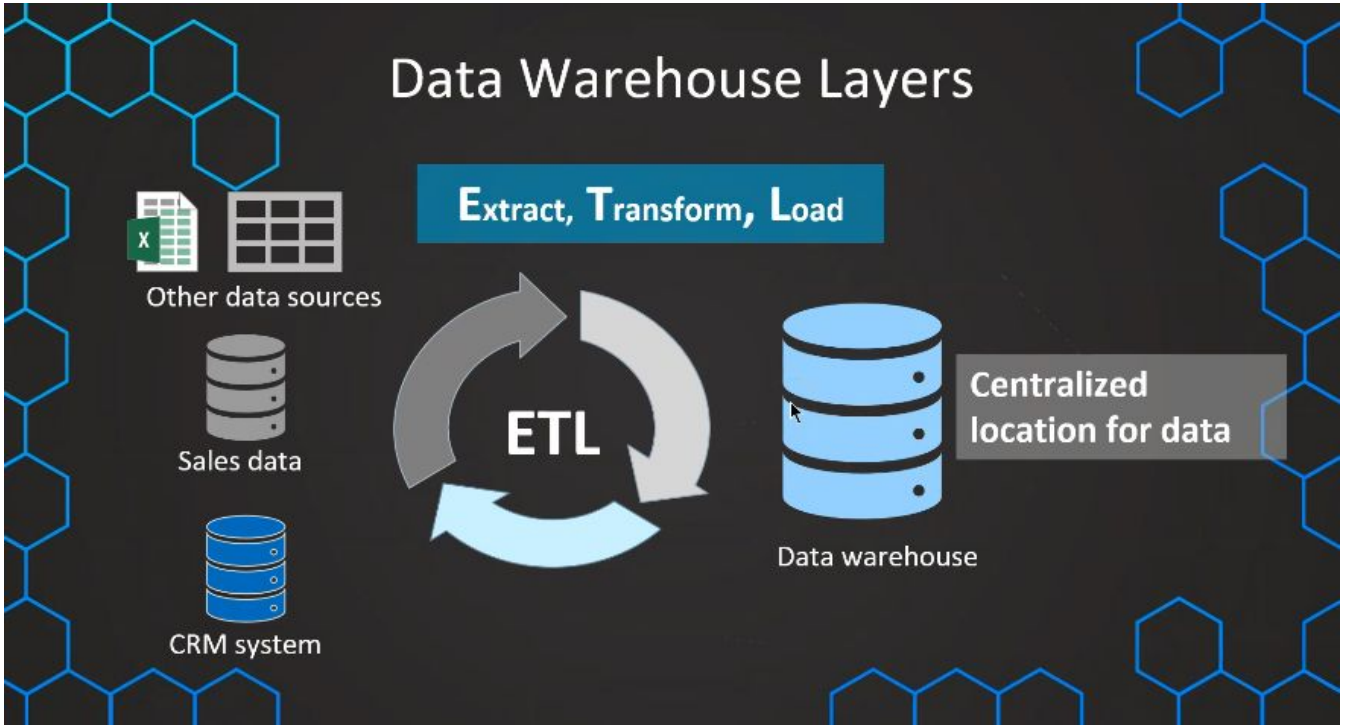
Slowly Changing Dimensions (SCD) - Type 3 (Additional Attributes)



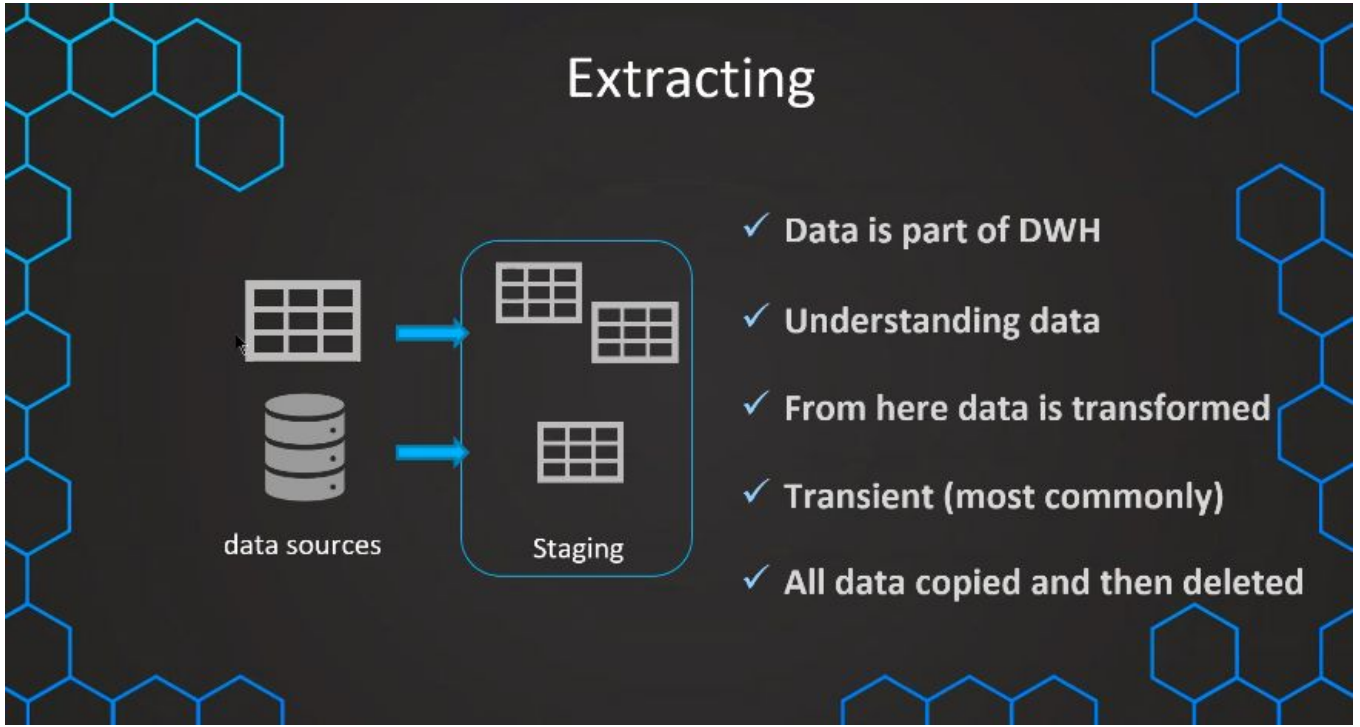
Slowly Changing Dimensions (SCD) - Type 3 (Additional Attributes)



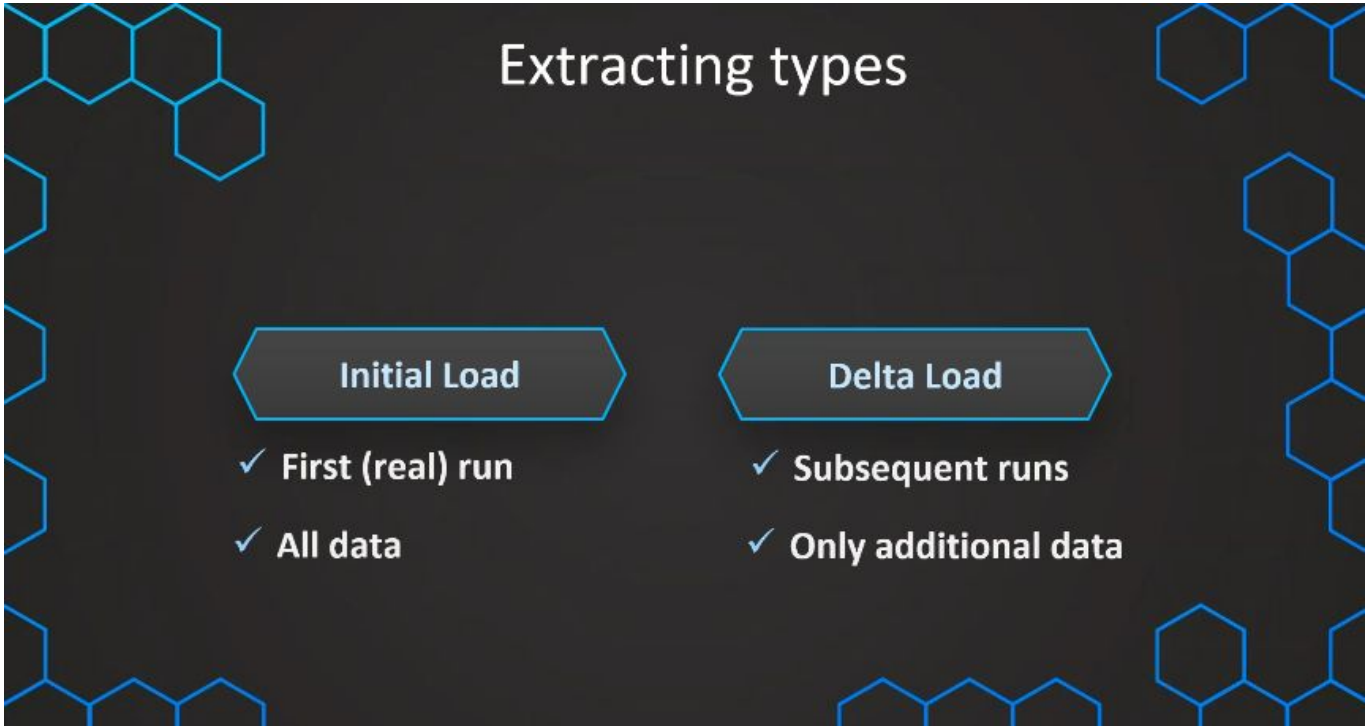
Extract-Transform-Load (ETL)



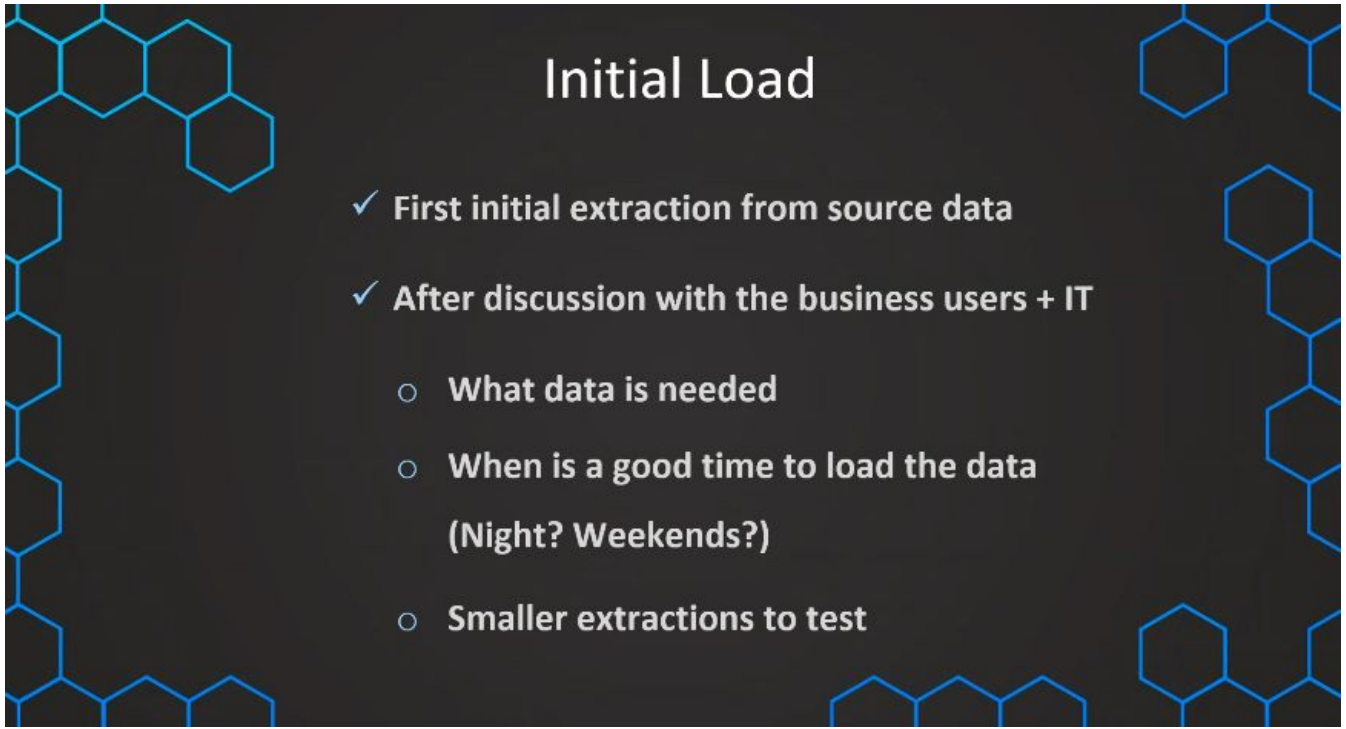
Extract-Transform-Load (ETL)



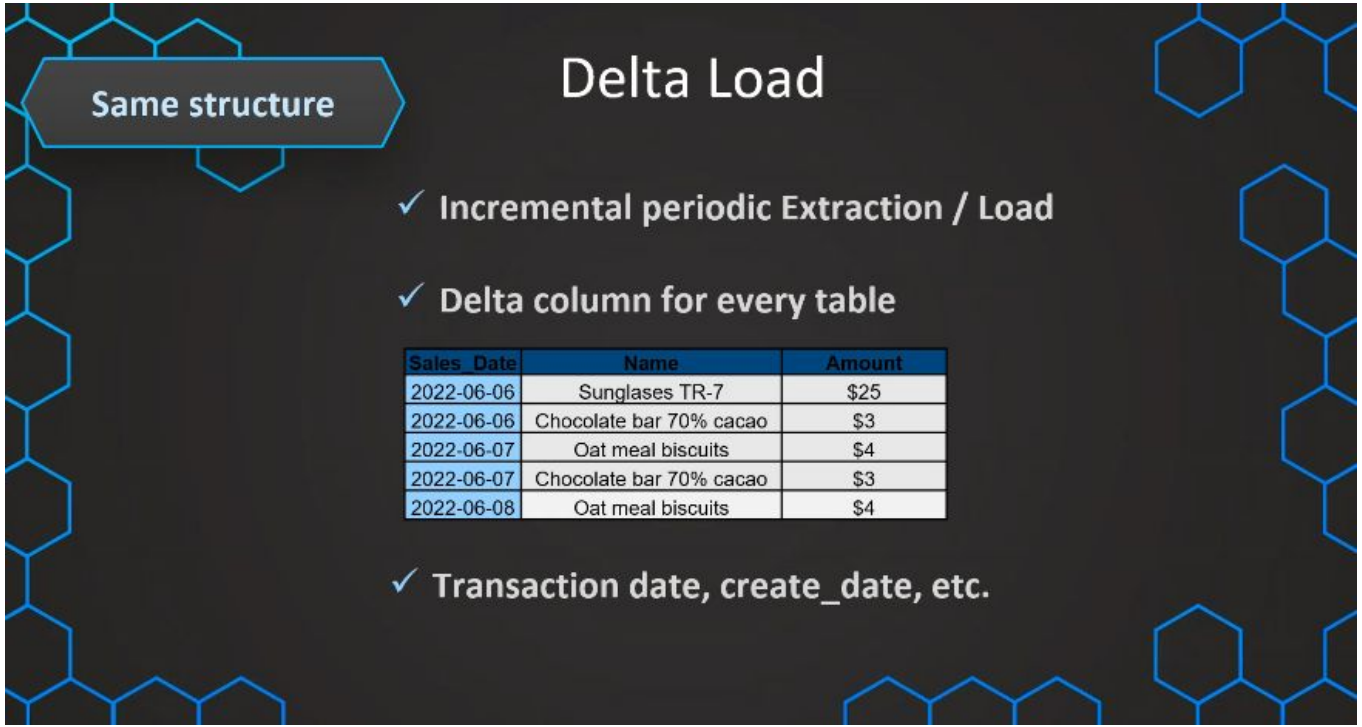
Extract-Transform-Load (ETL)



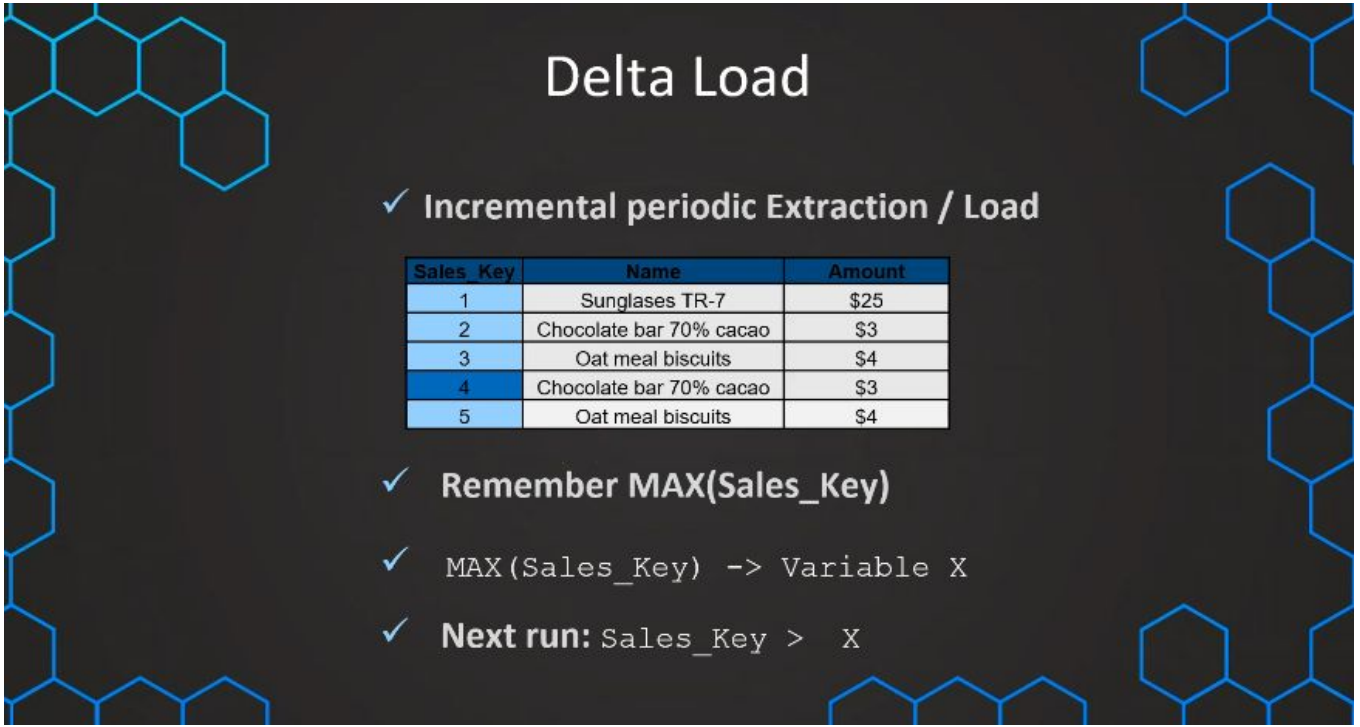
Extract-Transform-Load (ETL) - Initial Load



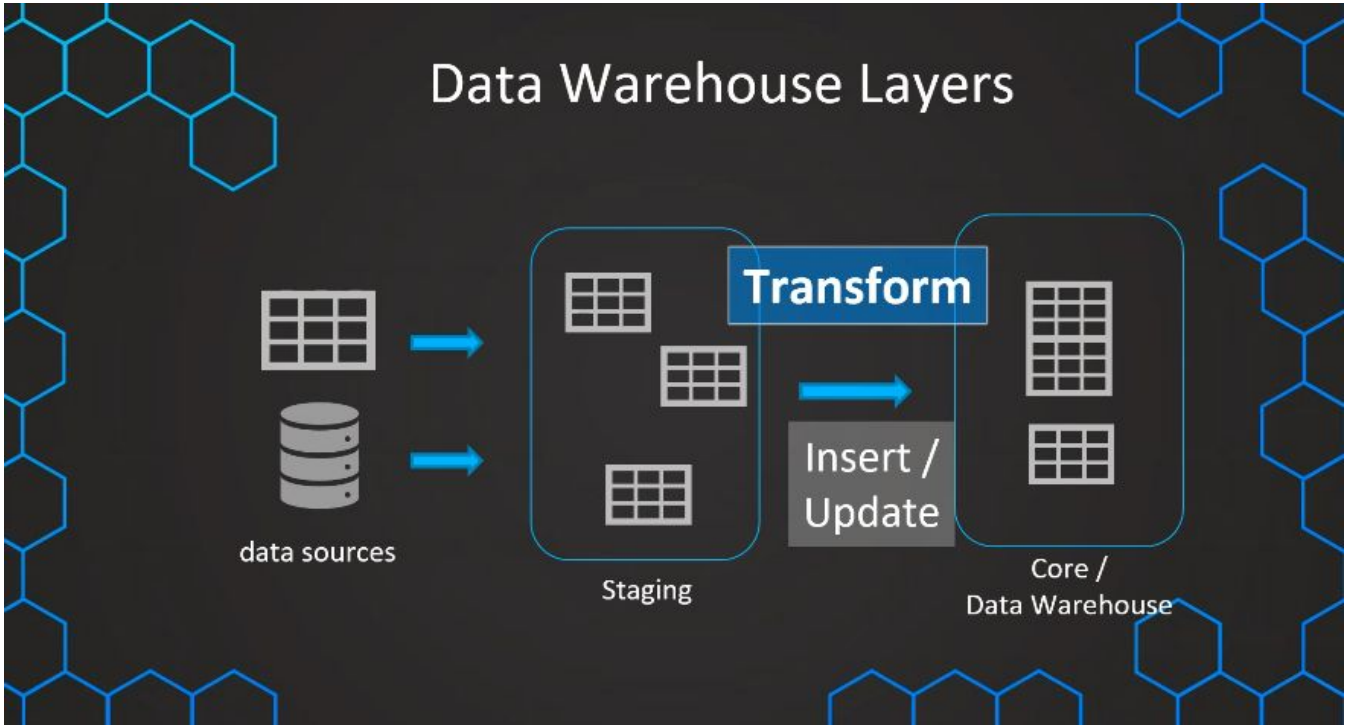
Extract-Transform-Load (ETL) - Delta Load



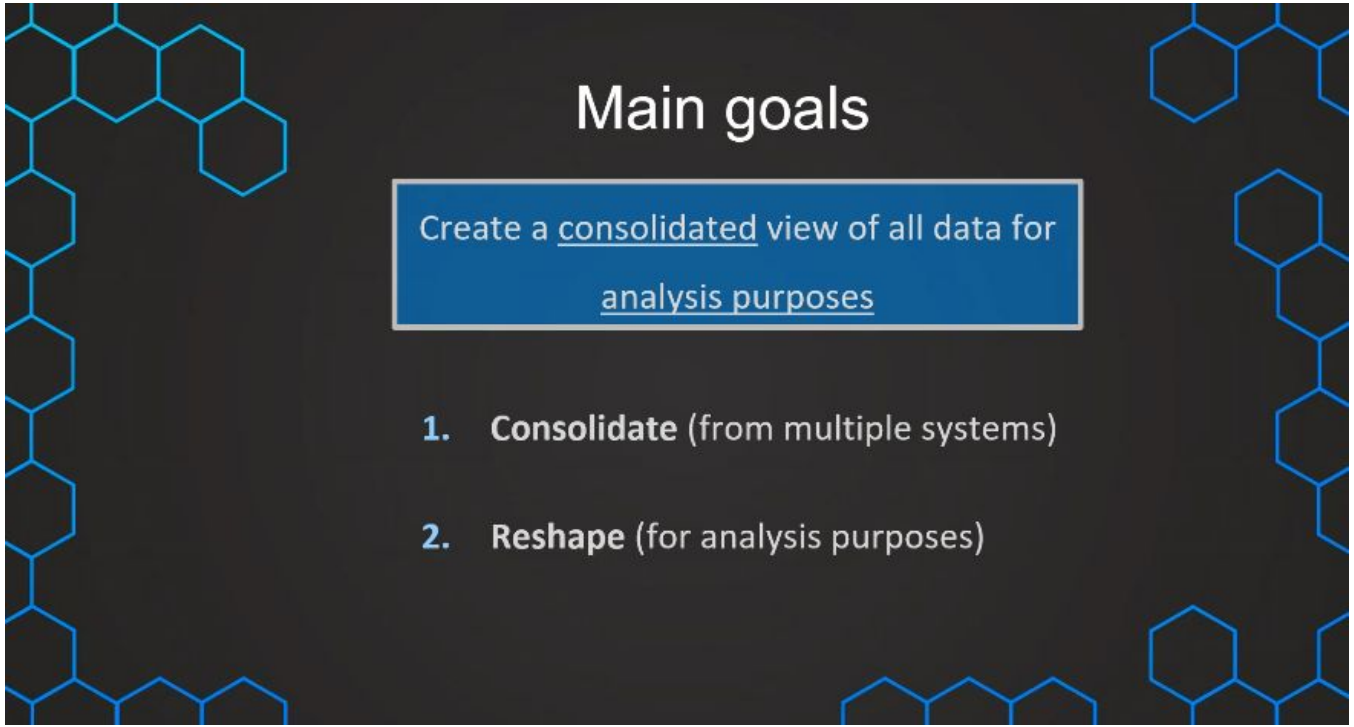
Extract-Transform-Load (ETL) - Delta Load



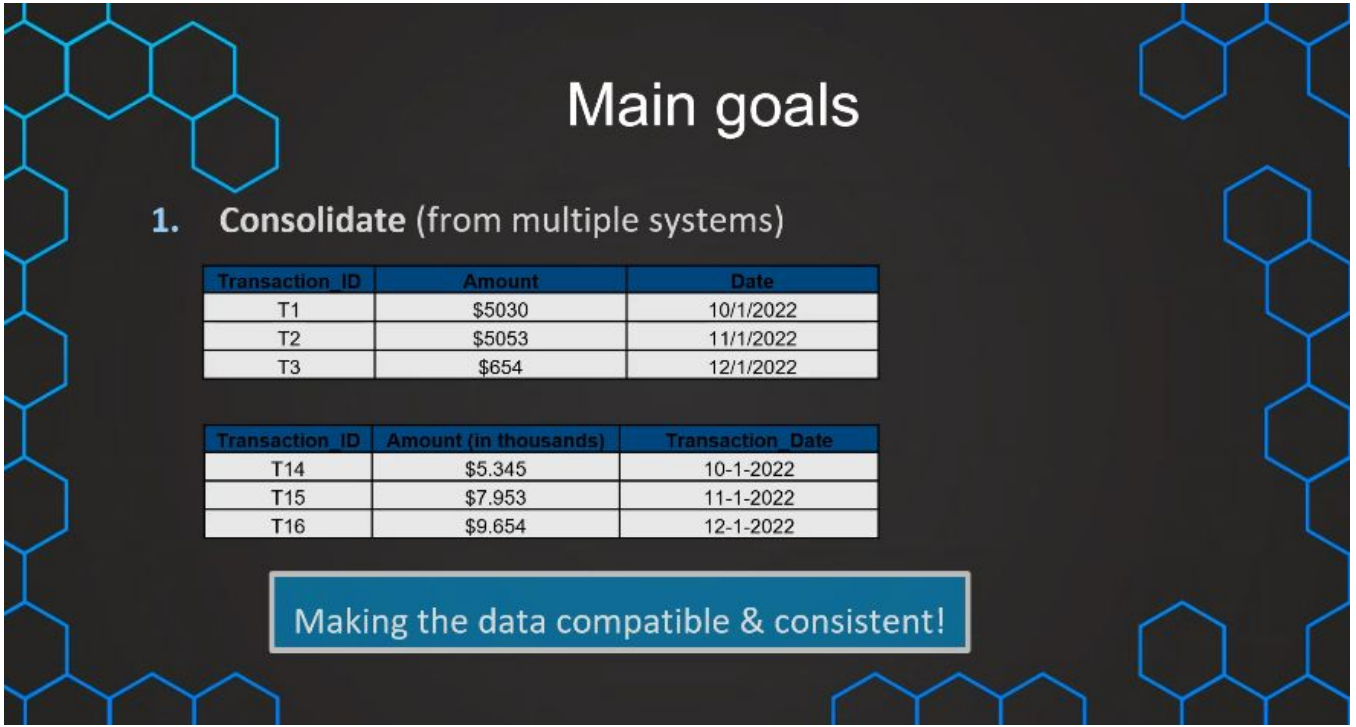
Extract-Transform-Load (ETL)



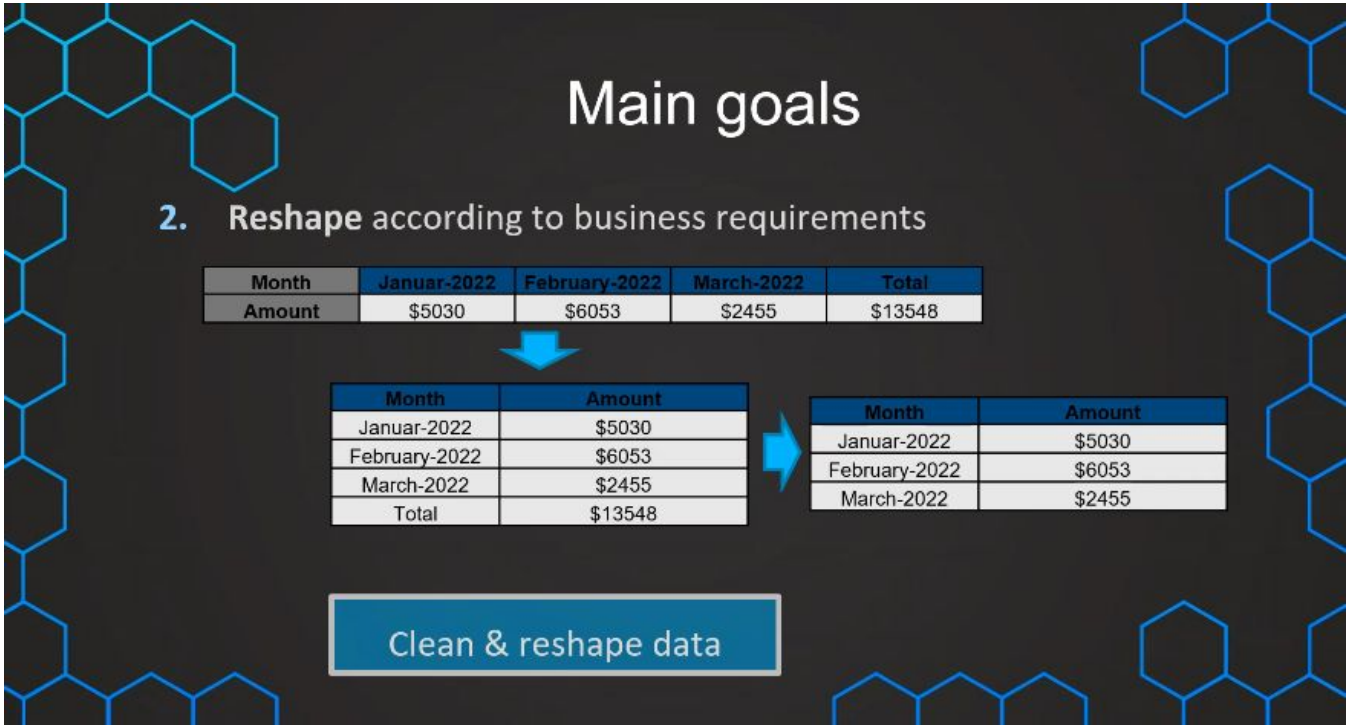
Extract-Transform-Load (ETL)



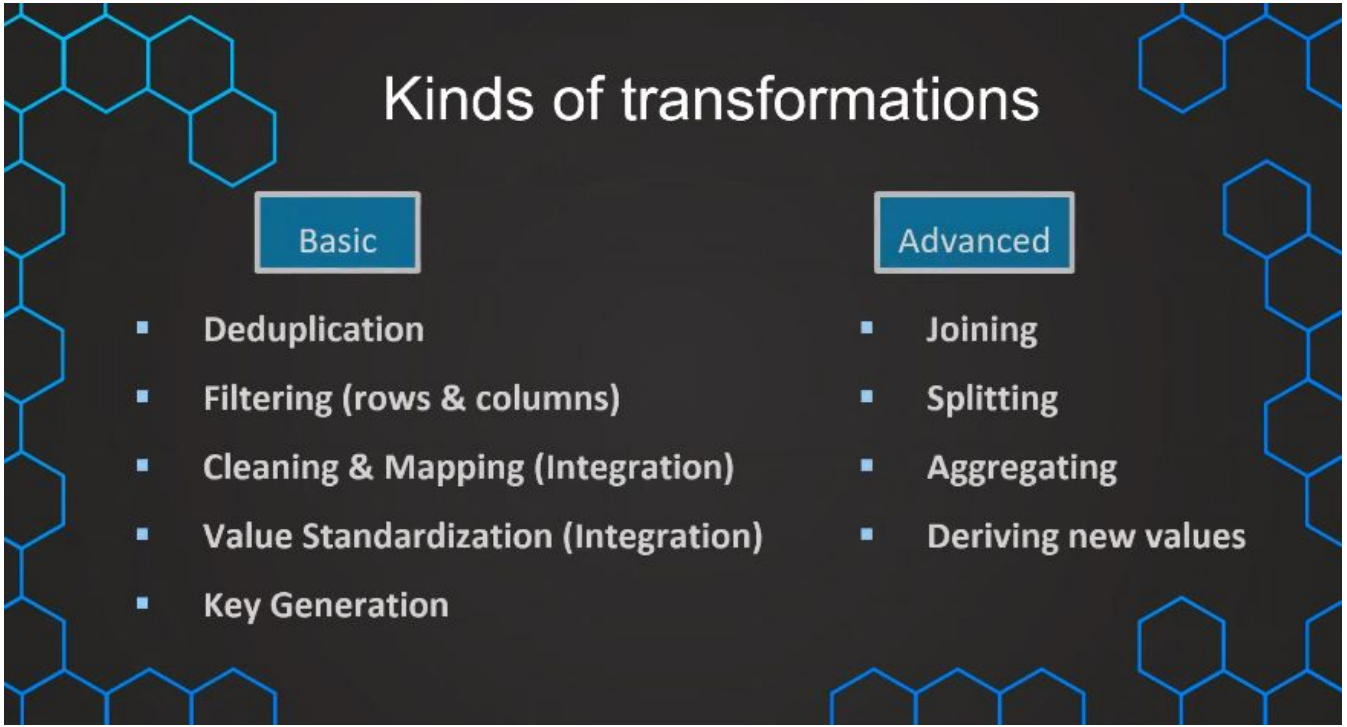
Extract-Transform-Load (ETL)



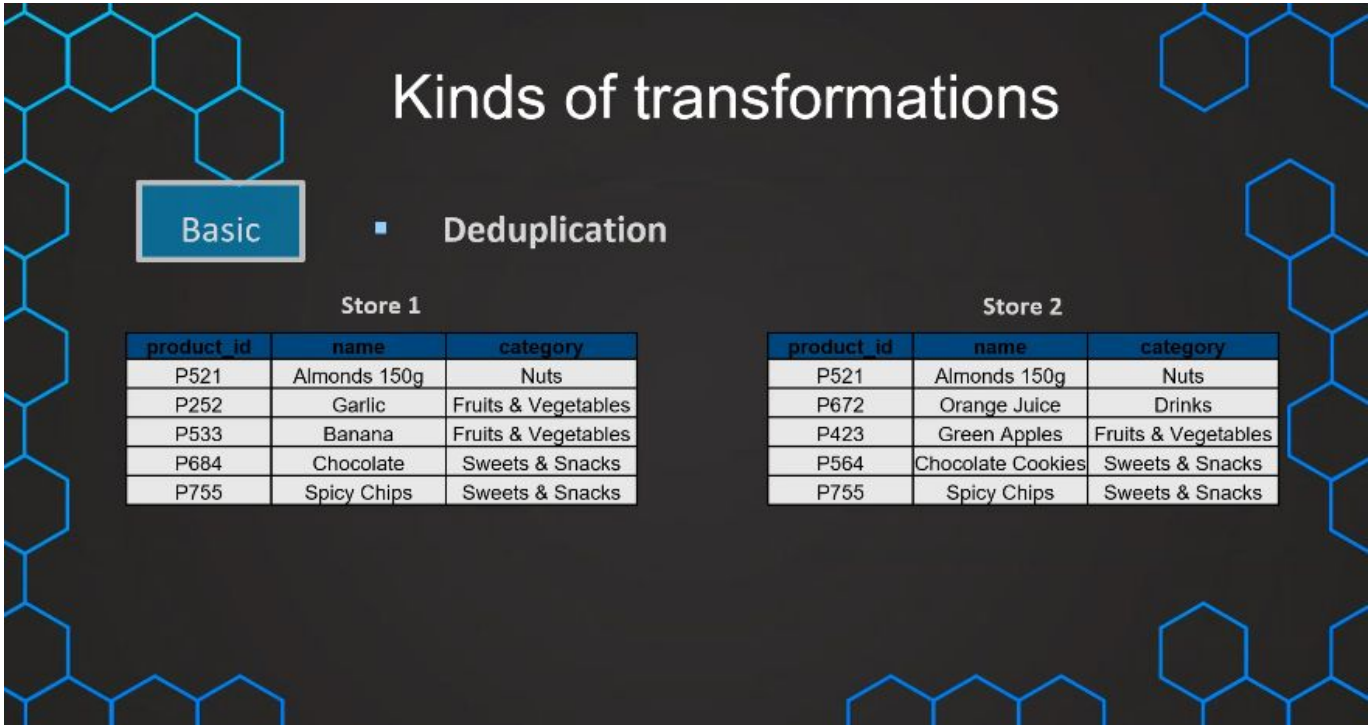
Extract-Transform-Load (ETL)



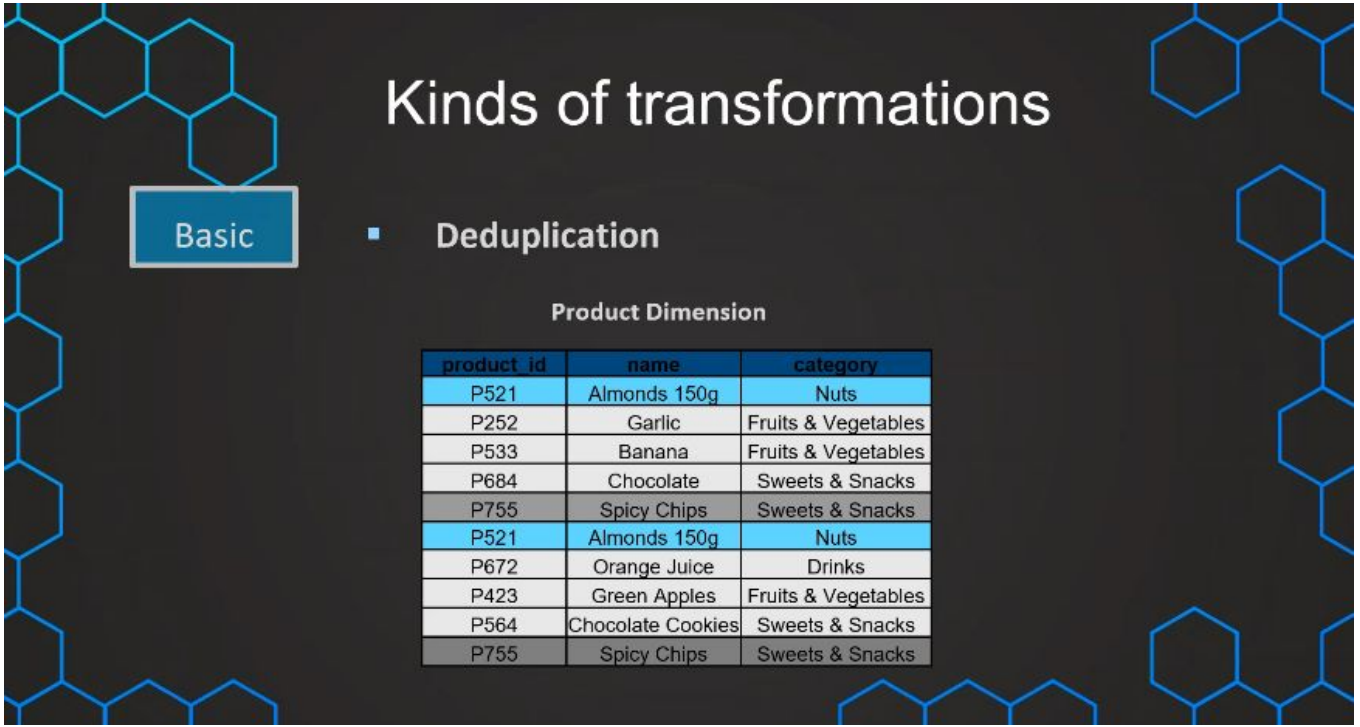
Extract-Transform-Load (ETL)



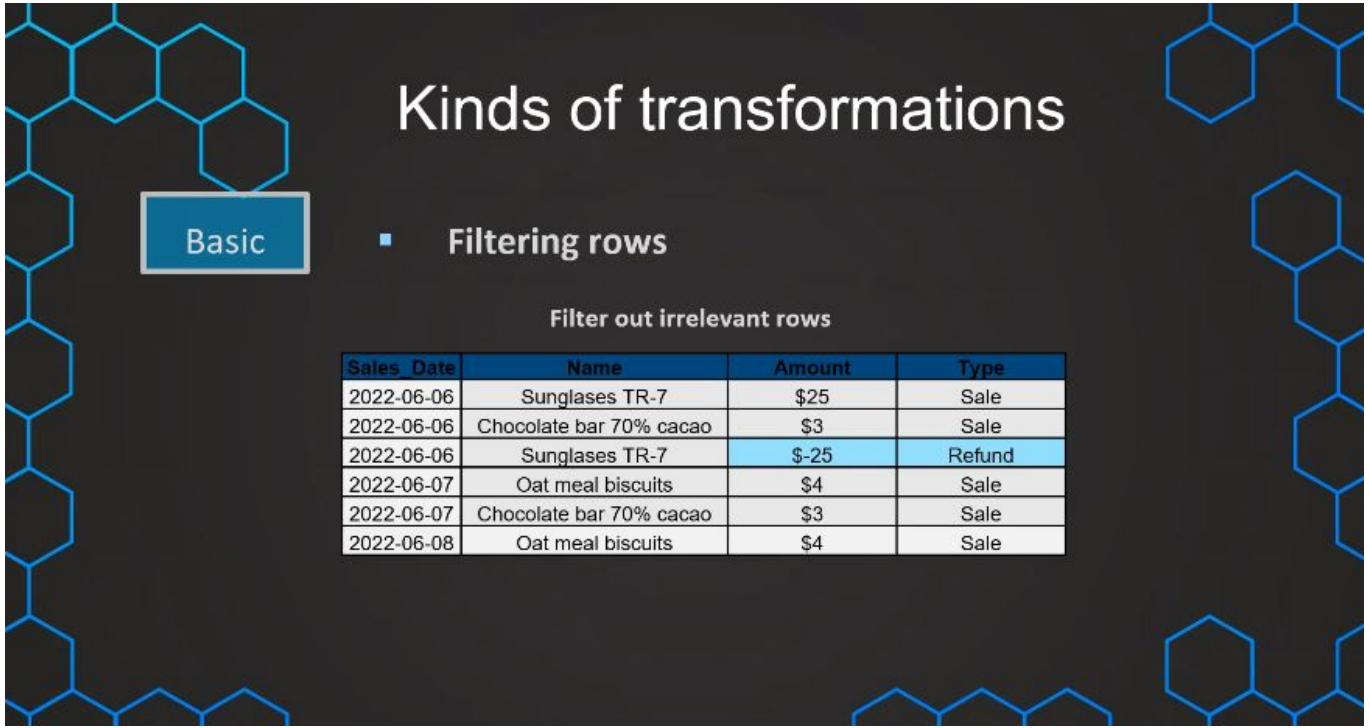
Extract-Transform-Load (ETL) - Deduplication



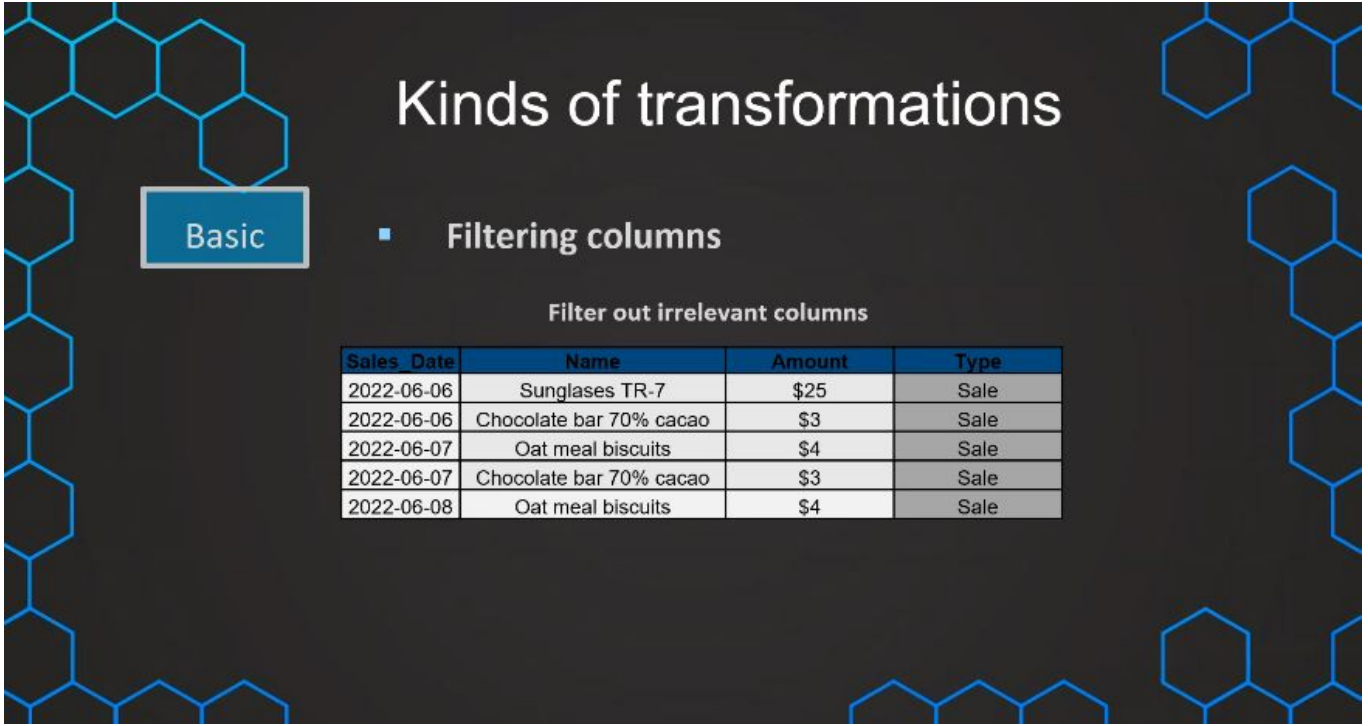
Extract-Transform-Load (ETL) - Deduplication



Extract-Transform-Load (ETL) - Filtering

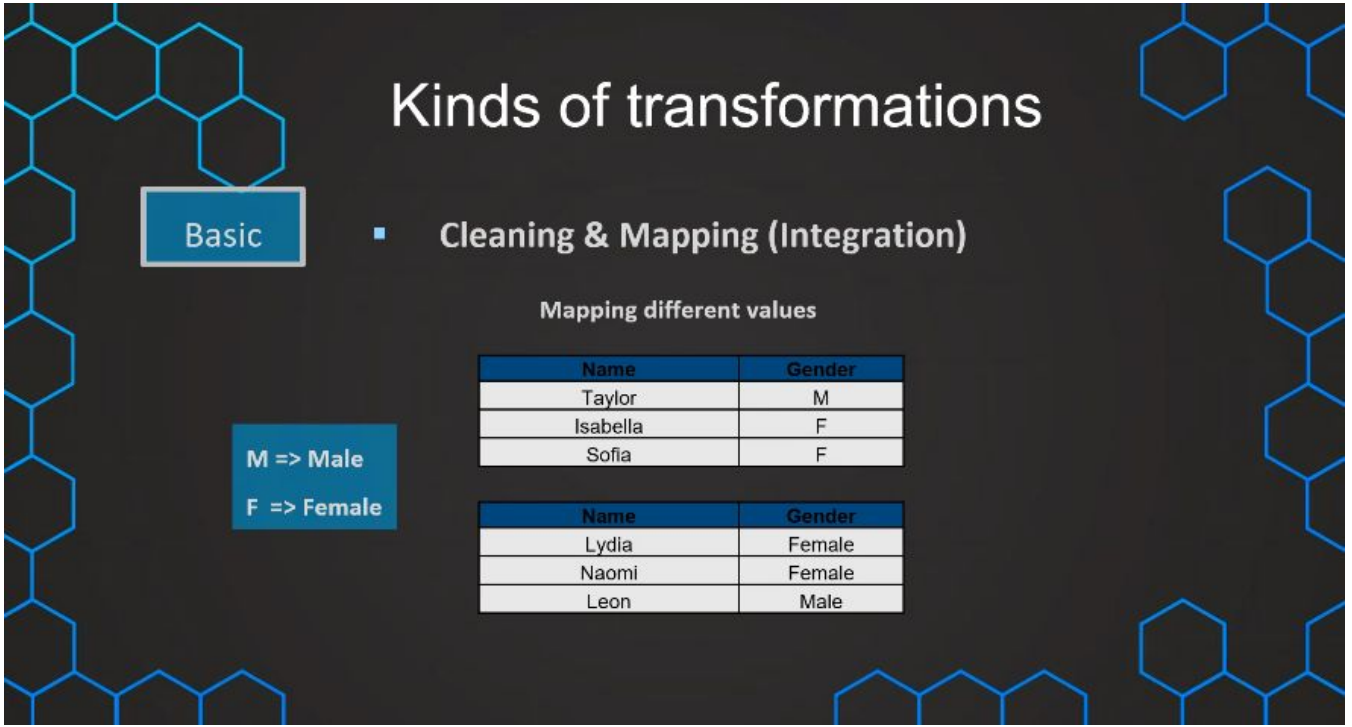


Extract-Transform-Load (ETL) - Filtering

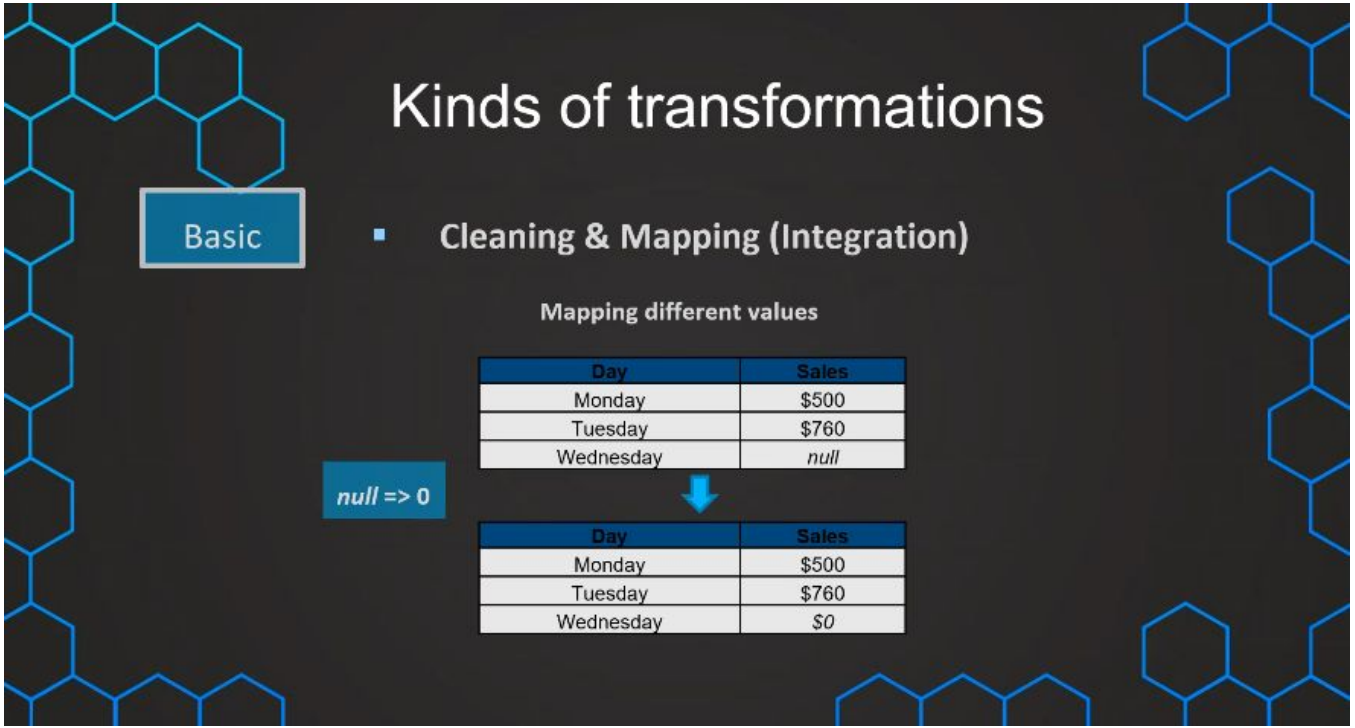


Sales_Date	Name	Amount	Type
2022-06-06	Sunglases TR-7	$25	Sale
2022-06-06	Chocolate bar 70% cacao	$3	Sale
2022-06-07	Oat meal biscuits	$4	Sale
2022-06-07	Chocolate bar 70% cacao	$3	Sale
2022-06-08	Oat meal biscuits	$4	Sale

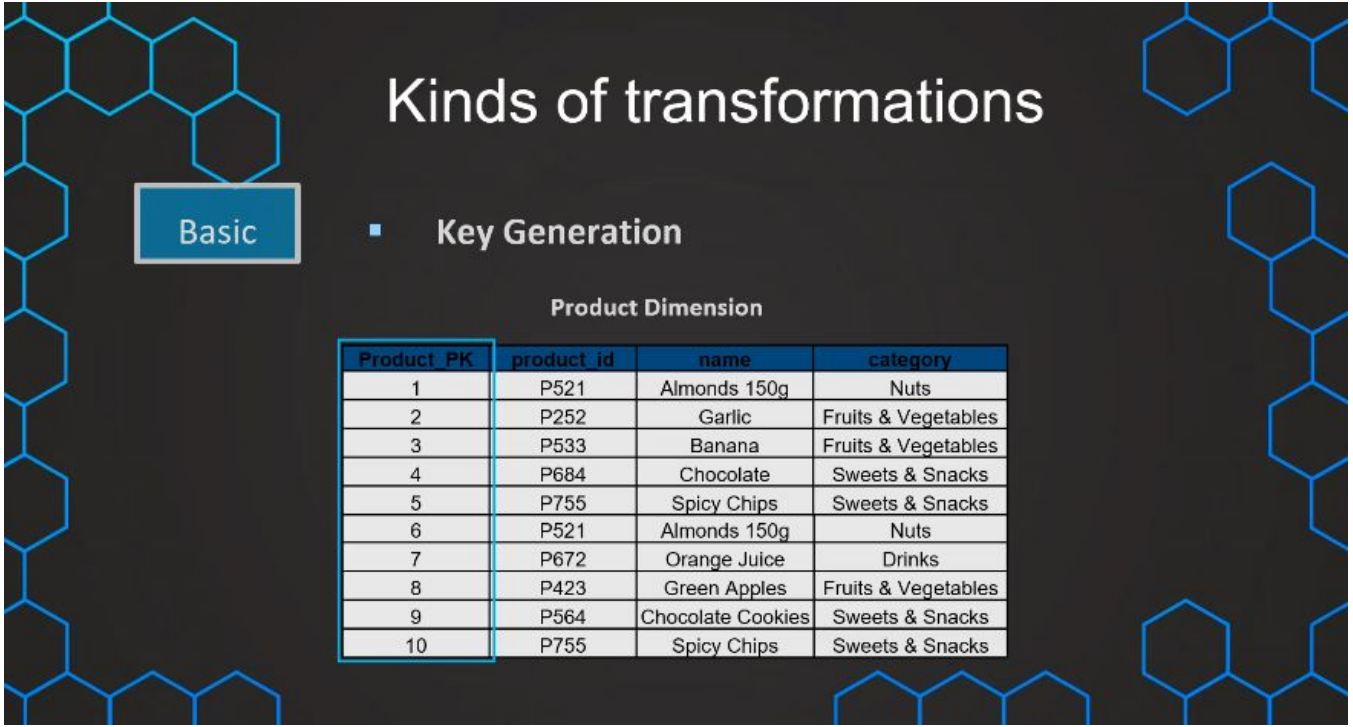
Extract-Transform-Load (ETL) - Cleaning & Mapping



Extract-Transform-Load (ETL) - Cleaning & Mapping

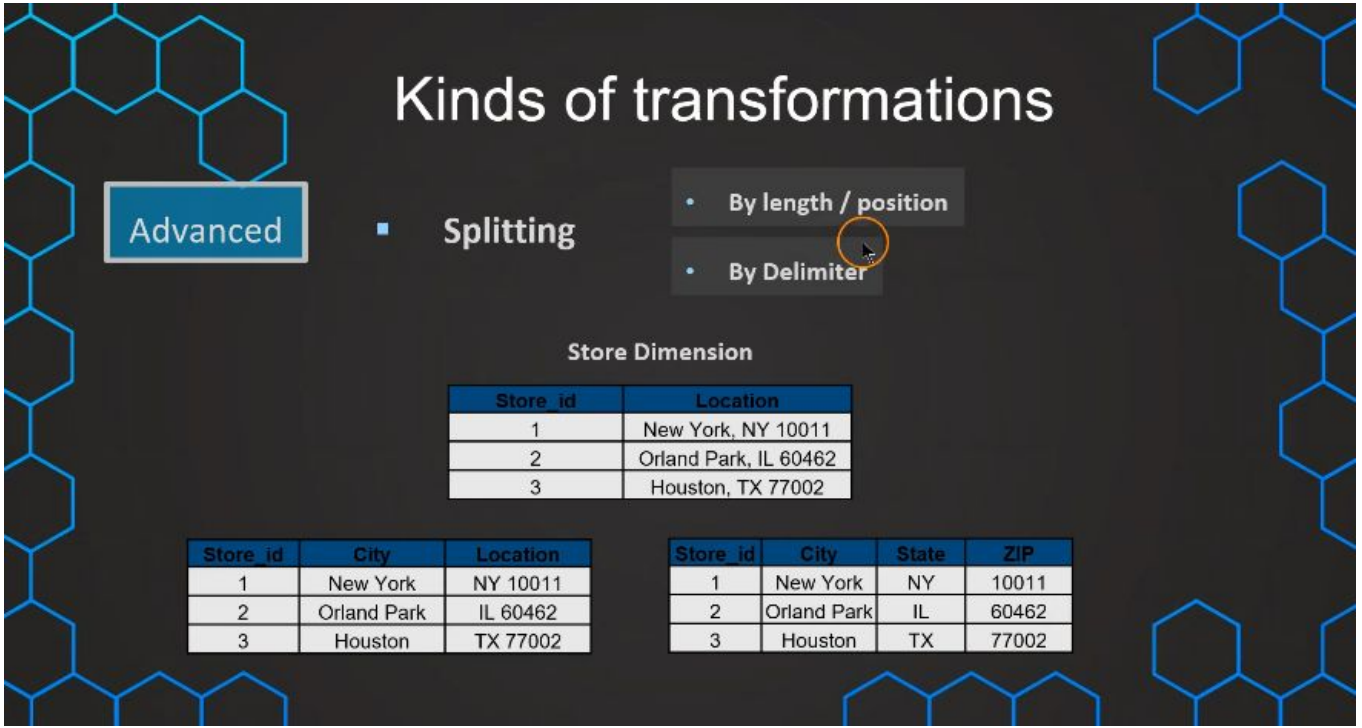


Extract-Transform-Load (ETL) - Key Generation

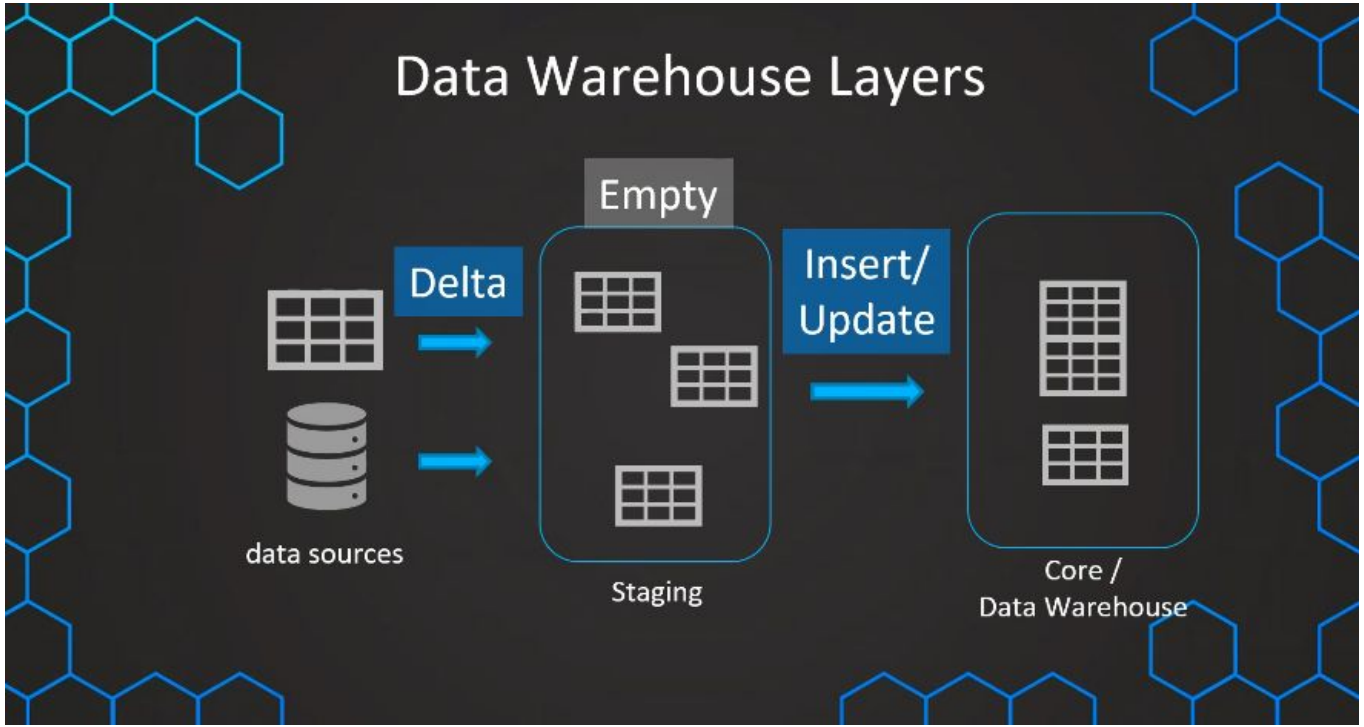


Product_PK	product_id	name	category
1	P521	Almonds 150g	Nuts
2	P252	Garlic	Fruits & Vegetables
3	P533	Banana	Fruits & Vegetables
4	P684	Chocolate	Sweets & Snacks
5	P755	Spicy Chips	Sweets & Snacks
6	P521	Almonds 150g	Nuts
7	P672	Orange Juice	Drinks
8	P423	Green Apples	Fruits & Vegetables
9	P564	Chocolate Cookies	Sweets & Snacks
10	P755	Spicy Chips	Sweets & Snacks

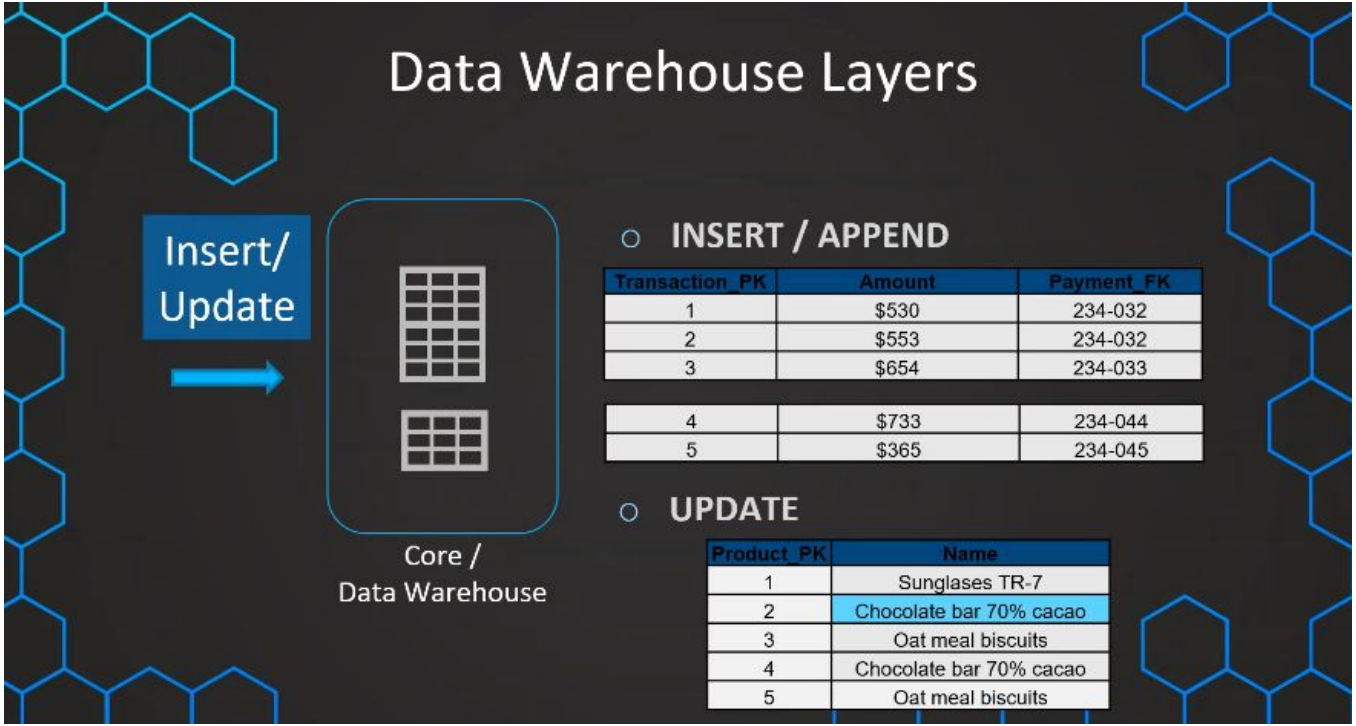
Extract-Transform-Load (ETL) - Splitting



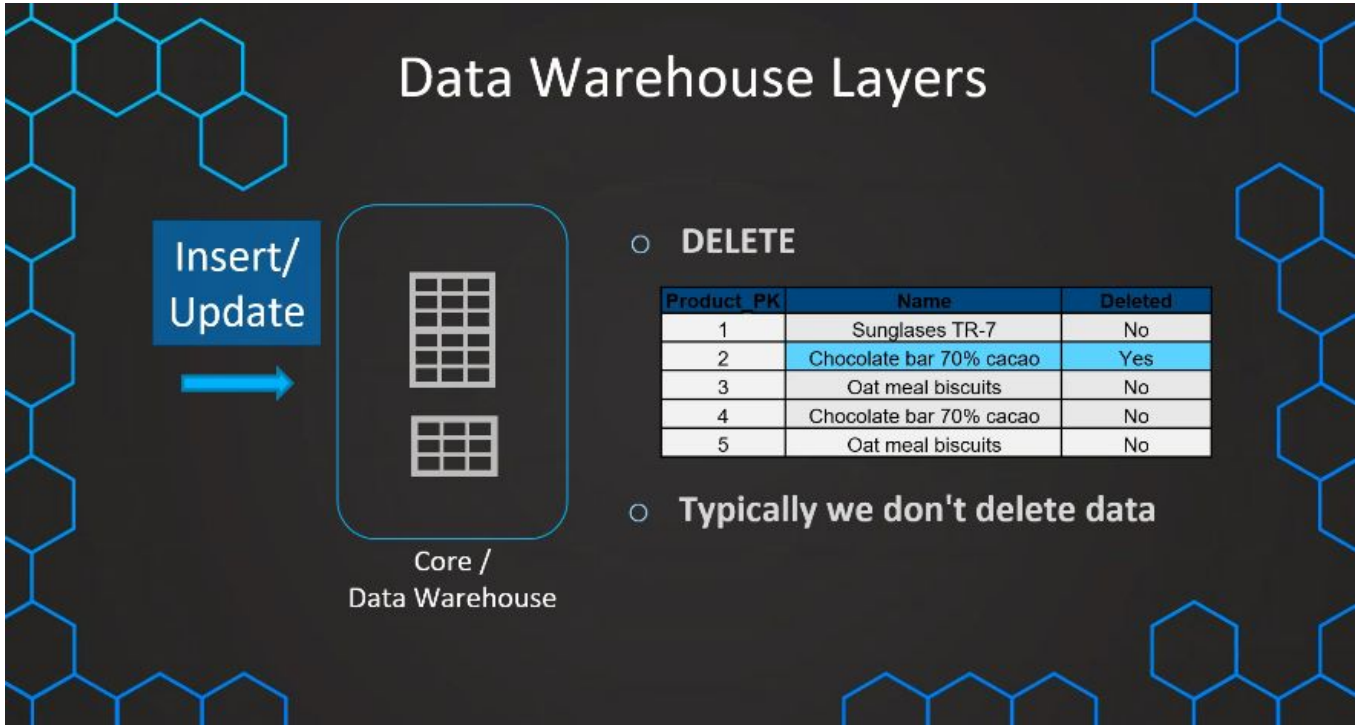
Extract-Transform-Load (ETL)



Extract-Transform-Load (ETL)



Extract-Transform-Load (ETL)



Product_PK	Name	Deleted
1	Sunglases TR-7	No
2	Chocolate bar 70% cacao	Yes
3	Oat meal biscuits	No
4	Chocolate bar 70% cacao	No
5	Oat meal biscuits	No

Extract-Transform-Load (ETL)



Extract

Retrieves and verifies data from various sources

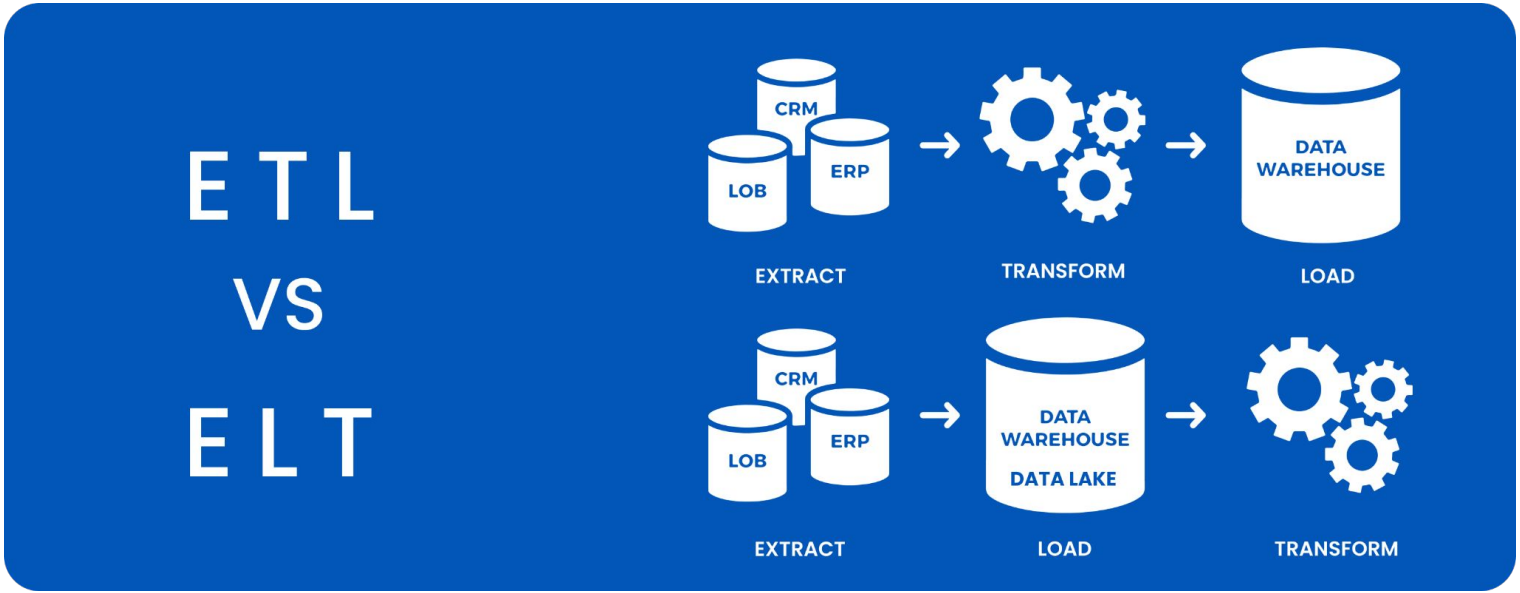
Transform

Processes and organizes extracted data so it is usable

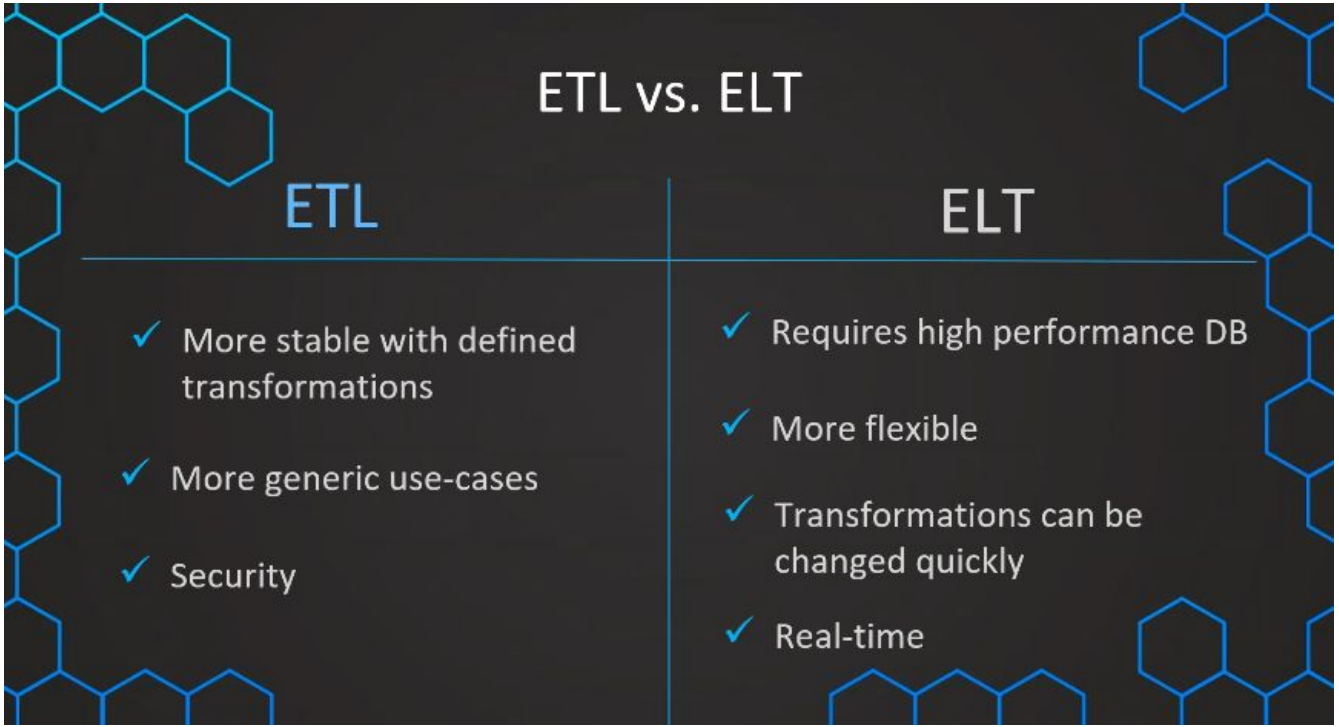
Load

Moves transformed data to a data repository

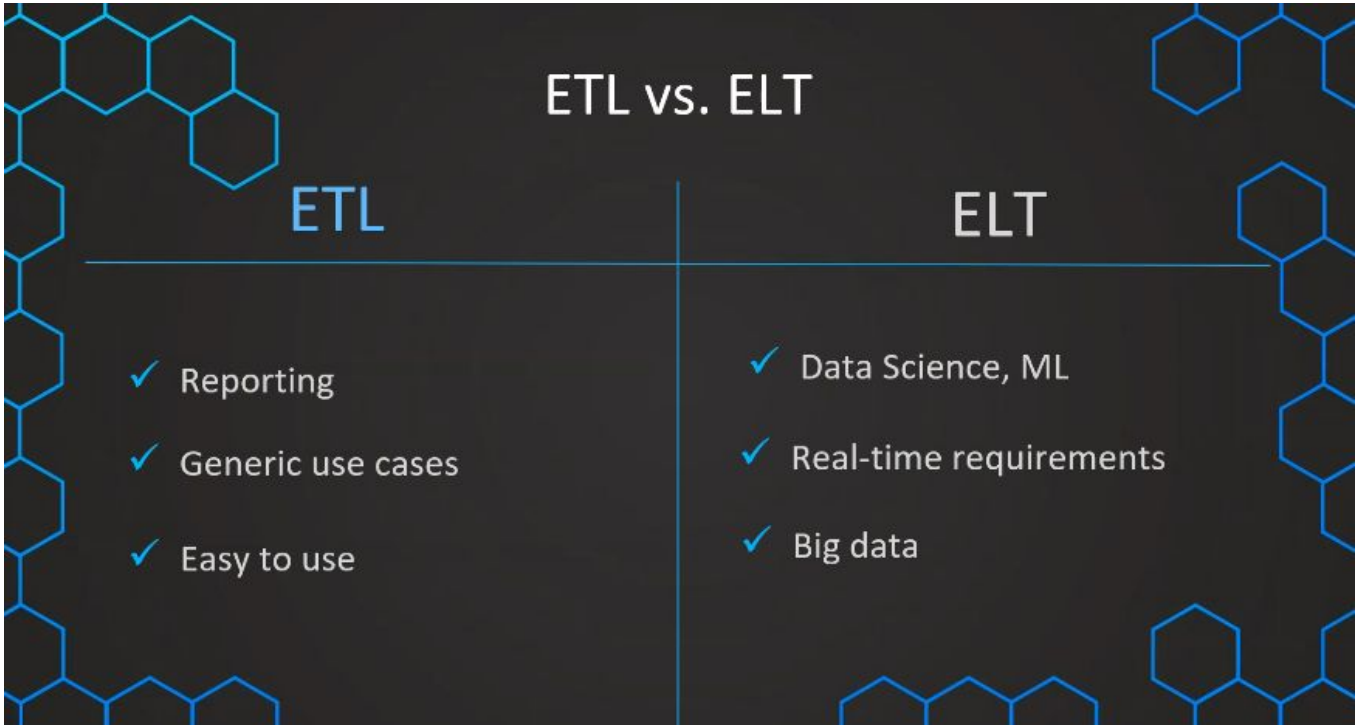
ETL vs. ELT


E T L
VS
E L T
CRM
LOB
ERP
EXTRACT
TRANSFORM
DATA WAREHOUSE
LOAD
CRM
LOB
ERP
EXTRACT
DATA WAREHOUSE
DATA LAKE
LOAD
TRANSFORM

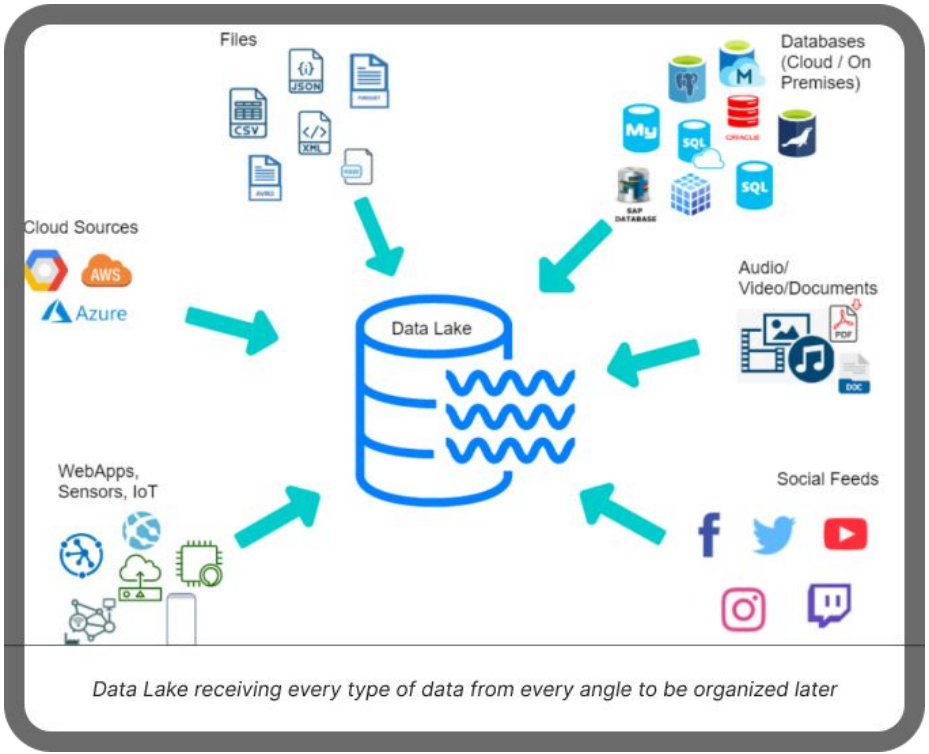
ETL vs. ELT


ETL vs. ELT
ETL
ELT
More stable with defined transformations
More generic use-cases
Security
Requires high performance DB
More flexible
Transformations can be changed quickly
Real-time

ETL vs. ELT


ETL vs. ELT
ETL
ELT
Reporting
Generic use cases
Easy to use
Data Science, ML
Real-time requirements
Big data

Data Lake



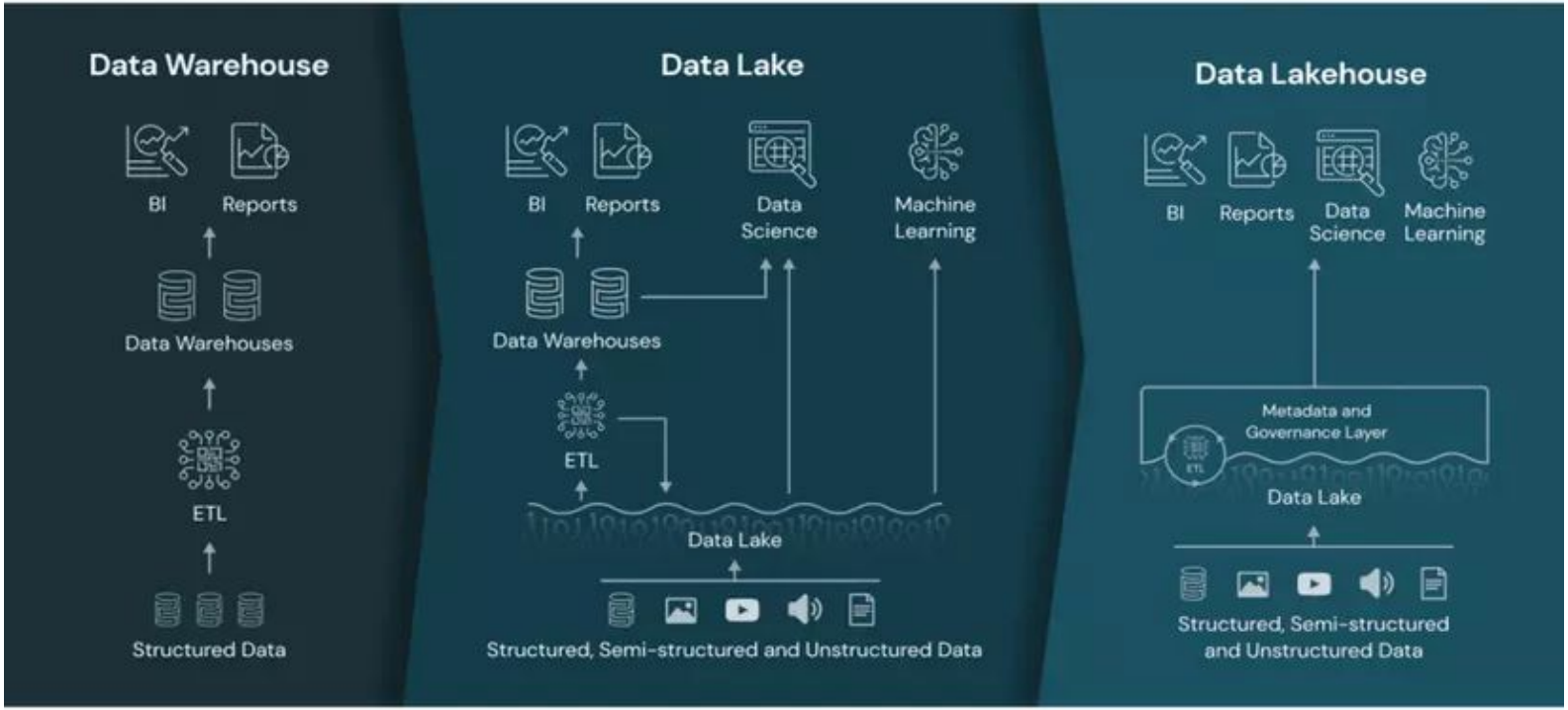
- Phát triển để ứng phó với sự bùng nổ dữ liệu lớn
- Lưu trữ dữ liệu bán cấu trúc (json), phi cấu trúc (image, audio) và có cấu trúc
- Đóng vai trò là kho lưu trữ linh hoạt
- Lợi ích:
  - Tính tập trung
  - Linh hoạt
  - Khả năng mở rộng
  - Chi phí thấp (Cloud)

Lake House

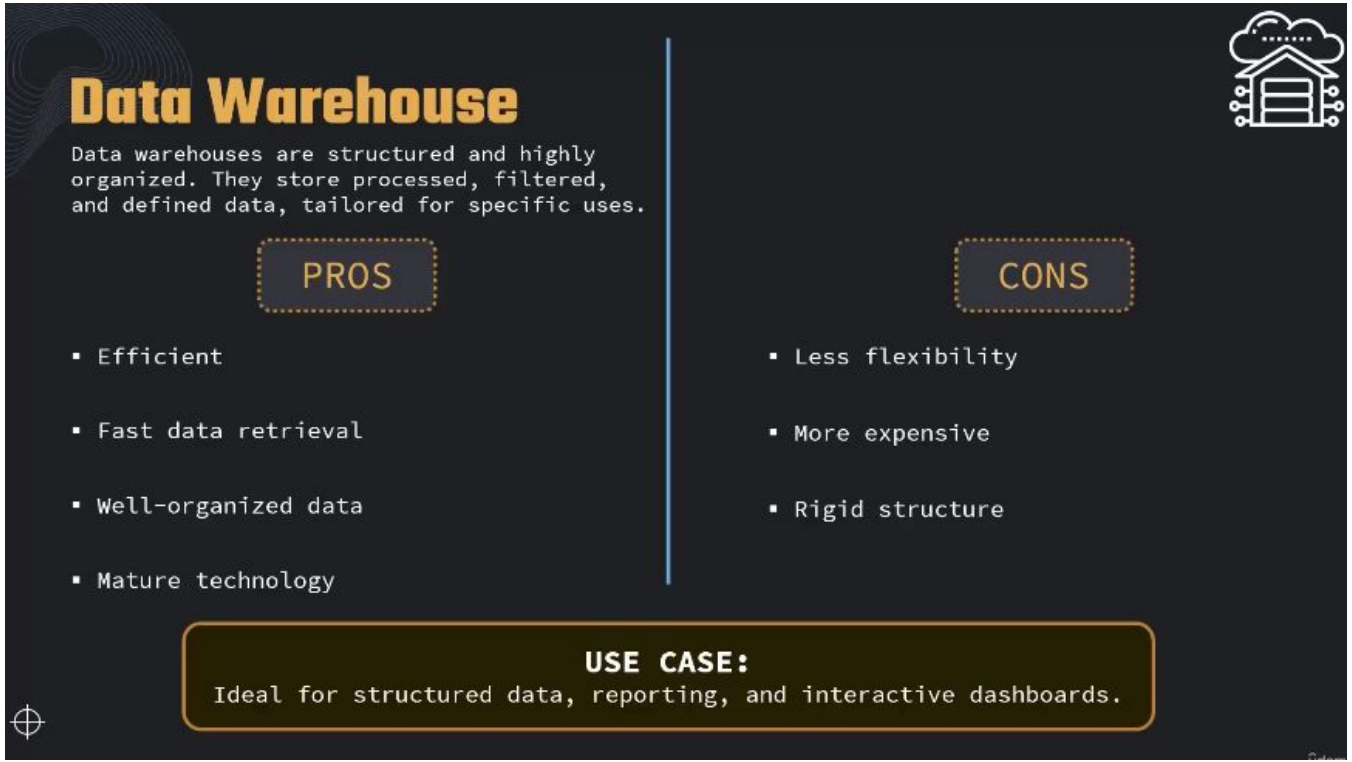


- Kết hợp Data Lake + Data Warehouse trong một kiến trúc duy nhất
- Hỗ trợ phân tích BI (SQL) và AI/ML (dữ liệu phi cấu trúc, bán cấu trúc)
- Đảm bảo ACID transaction thông qua:
  - Delta Table (Delta Lake), Apache Iceberg, Hudi
  - Cơ chế Write-Ahead Log + Snapshot → mọi thay đổi được ghi log trước khi commit
  - Dùng metadata & versioning để đảm bảo *Atomicity, Consistency, Isolation, Durability*
- Một nguồn dữ liệu duy nhất (single source of truth) cho phân tích & ML

Data Lake & Lake House



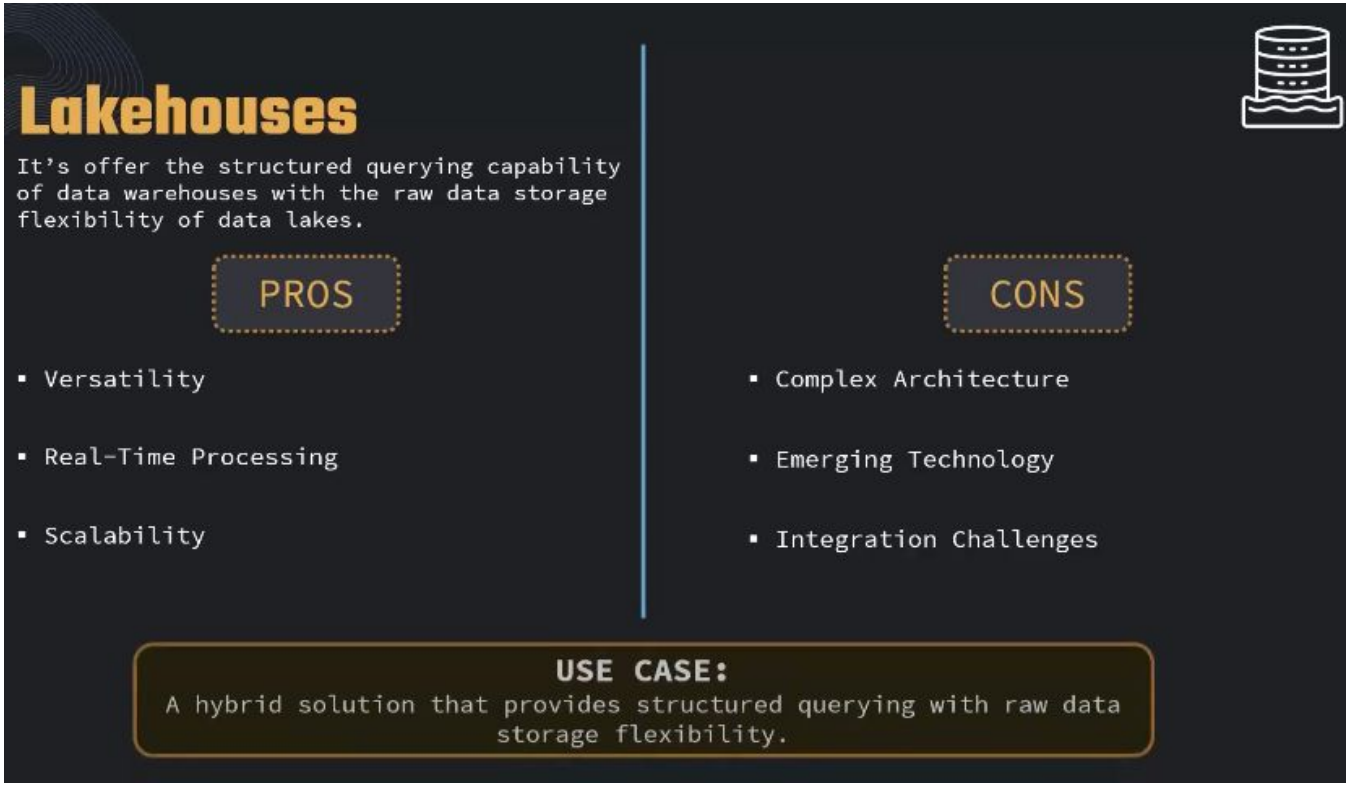
Data Warehouse & Data Lake & Lake House



Data Warehouse & Data Lake & Lake House



Data Warehouse & Data Lake & Lake House



Data Warehouse & Data Lake & Lake House



Data Lake

Data Swamp

ENSURE

- Strict Data Governance
- Quality Control
- Use Metadata Management
- Use Data Catalog



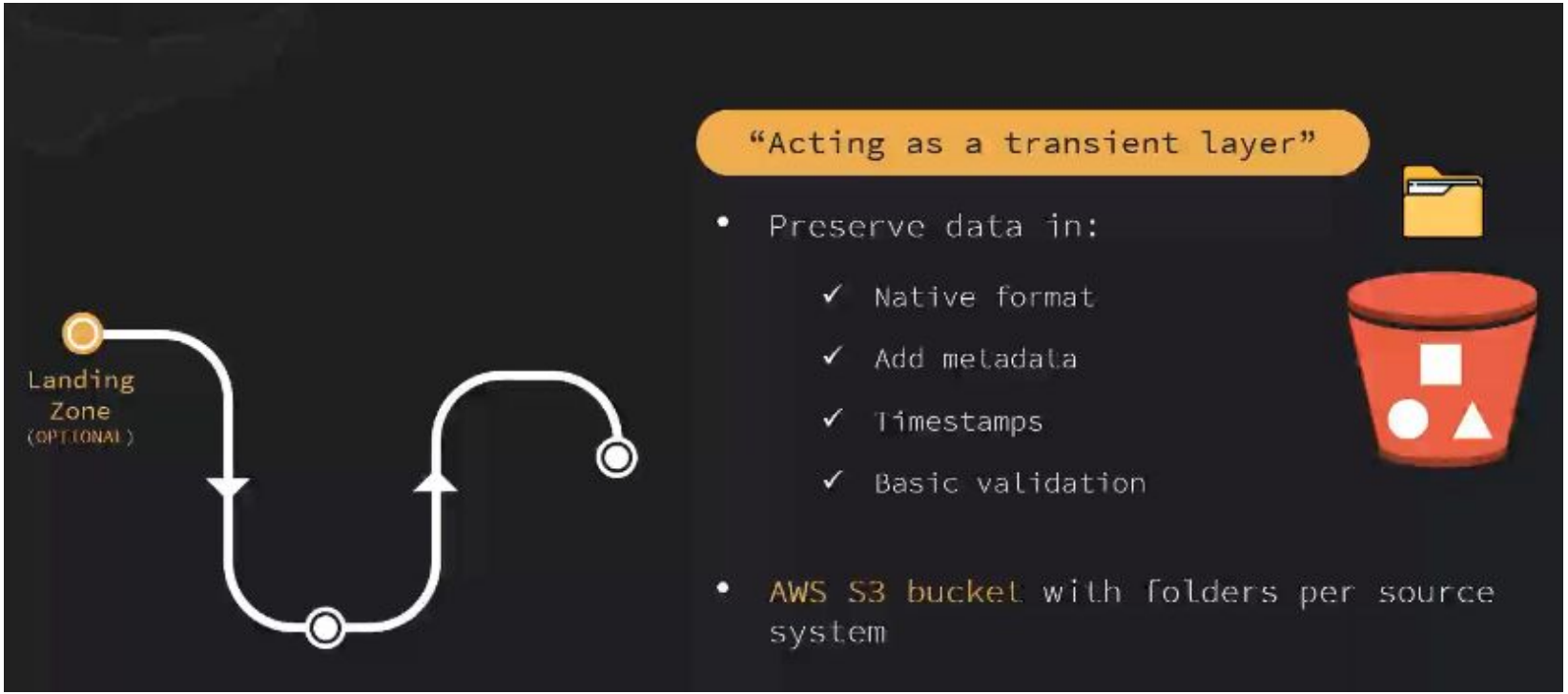
WITHOUT

- Adequate Metadata
- Quality Checks
- Organization

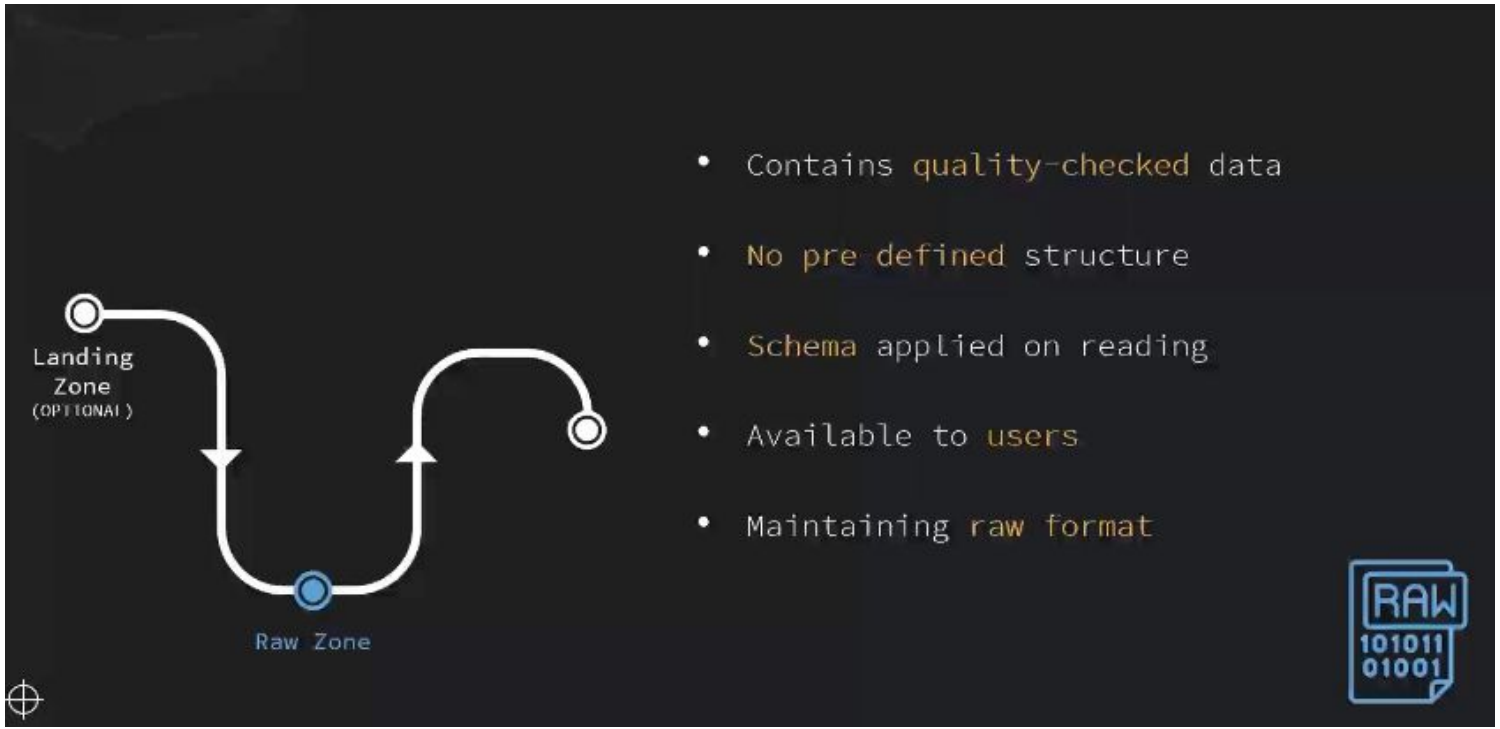
Different Zones in Data Lake

- **Single-layer** vs. **Multi-layer** (with multiple zones)
  - Single-layer is simple
- Multi-layer recommendation:
  - Structured organization
  - Quality control
  - Enhanced governance

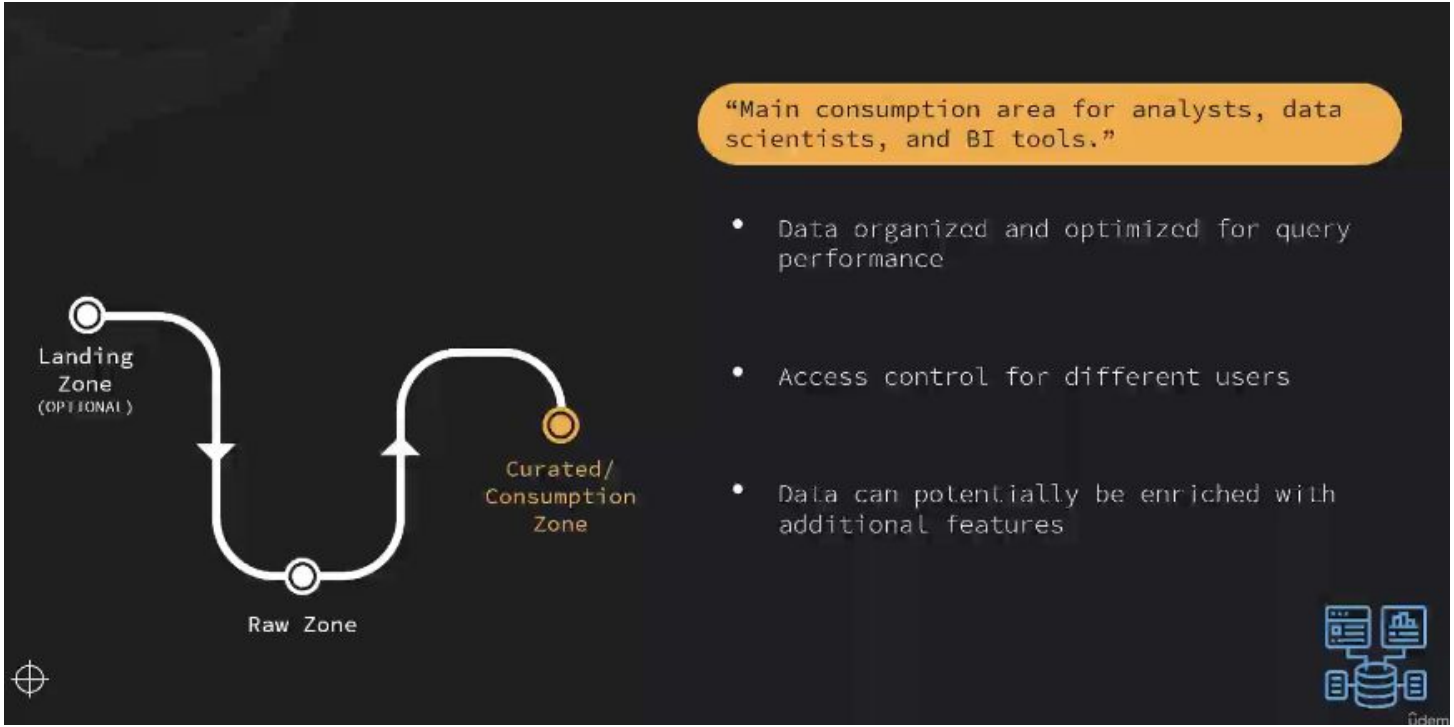
Different Zones in Data Lake - Landing Zone



Different Zones in Data Lake - Raw Zone



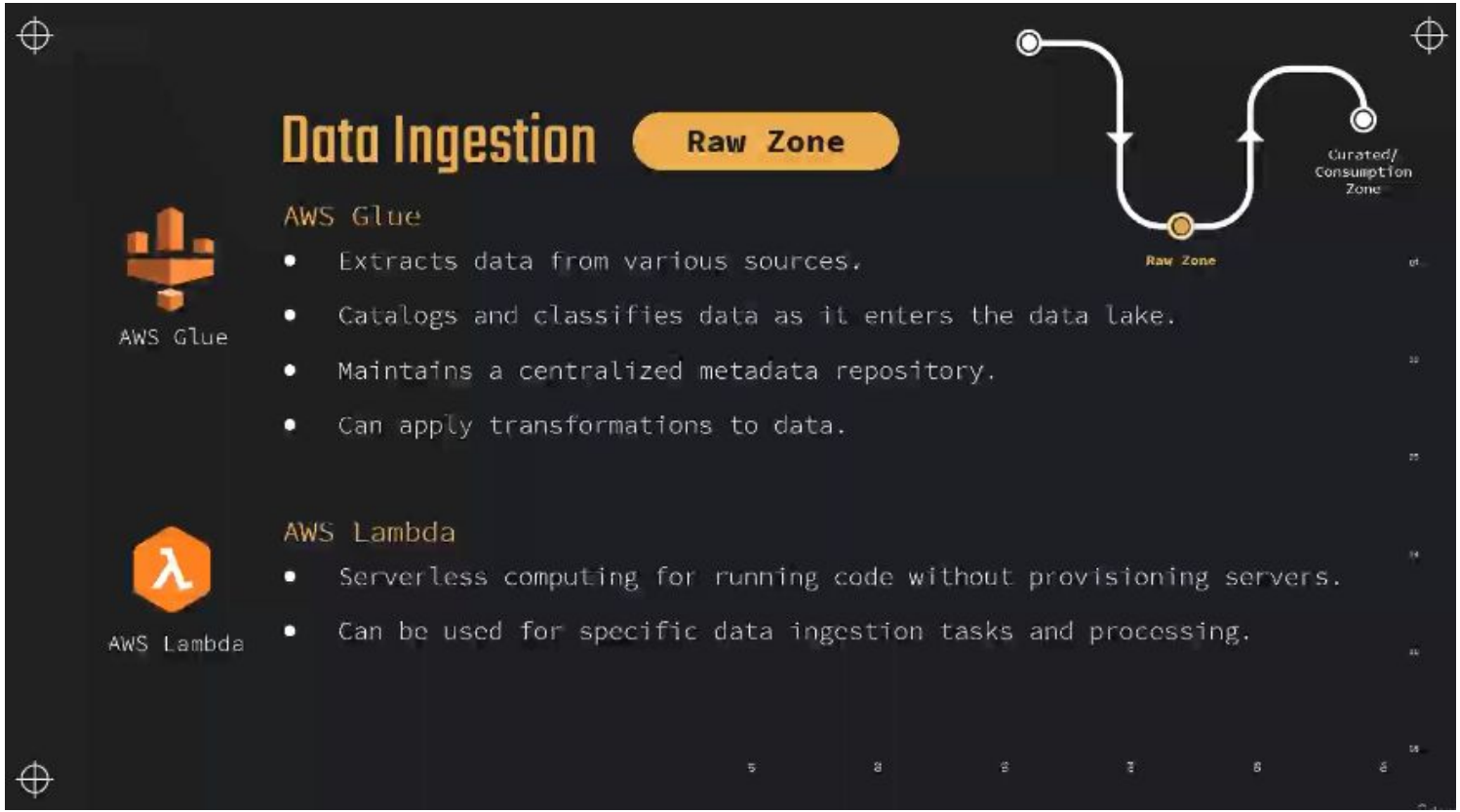
Different Zones in Data Lake - Landing Zone



Different Zones in Data Lake - Dev Zone



Tools for different zones in Data Lake



Tools for different zones in Data Lake

Query and Analysis

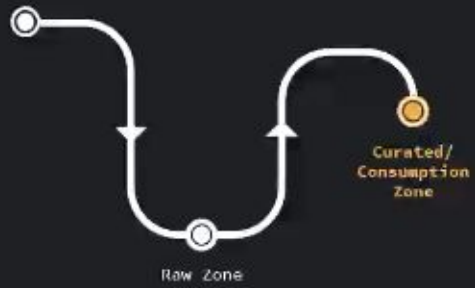
Curated/ Consumption Zone



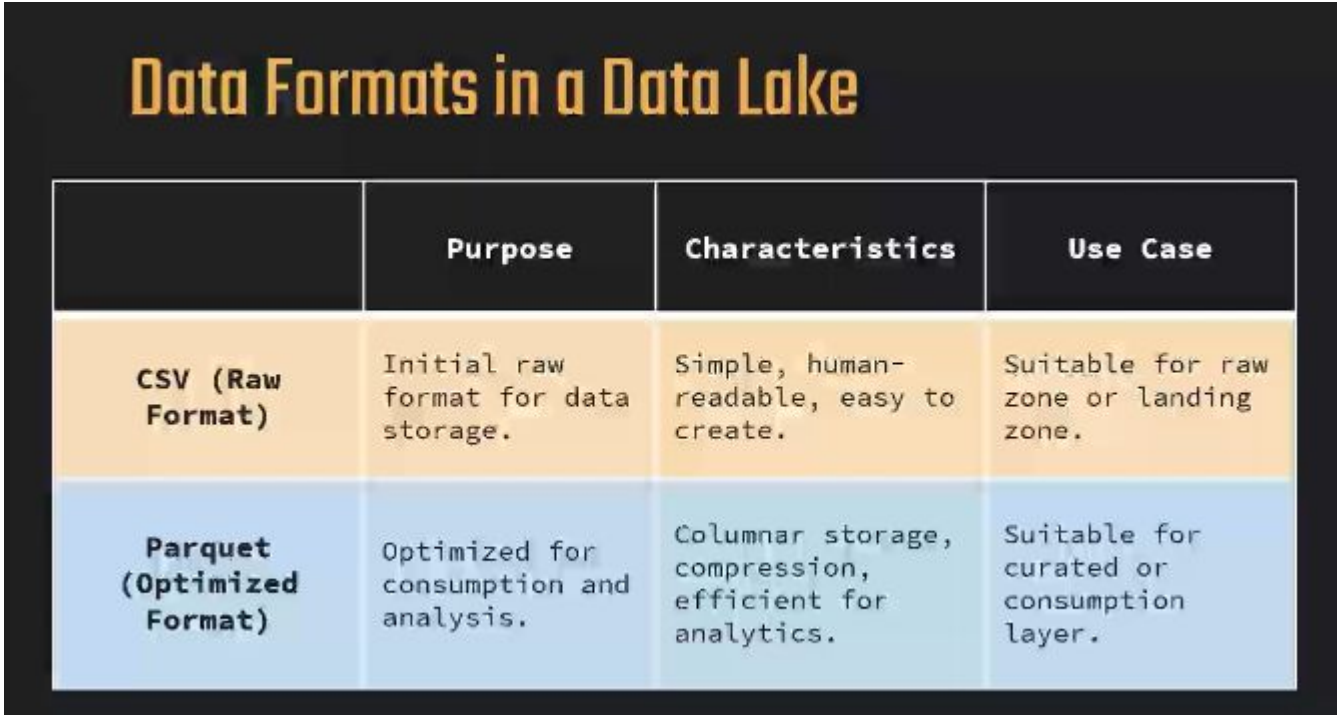
AWS Athena

Amazon Athena

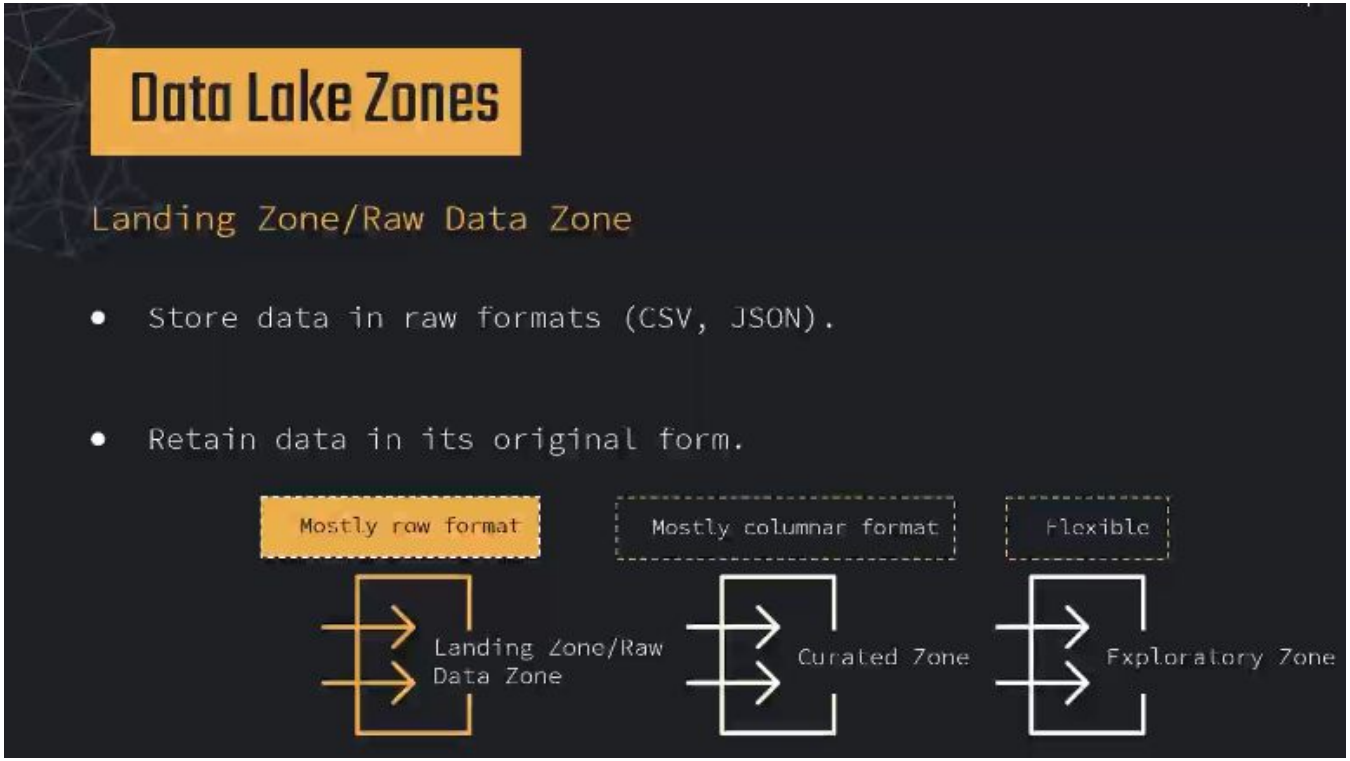
- Allows querying and analyzing data directly from S3 buckets.
- Ideal for ad hoc querying and exploratory data analysis.
- Provides quick SQL-like queries with good query performance.
- No need to load data into a separate database or data warehouse.



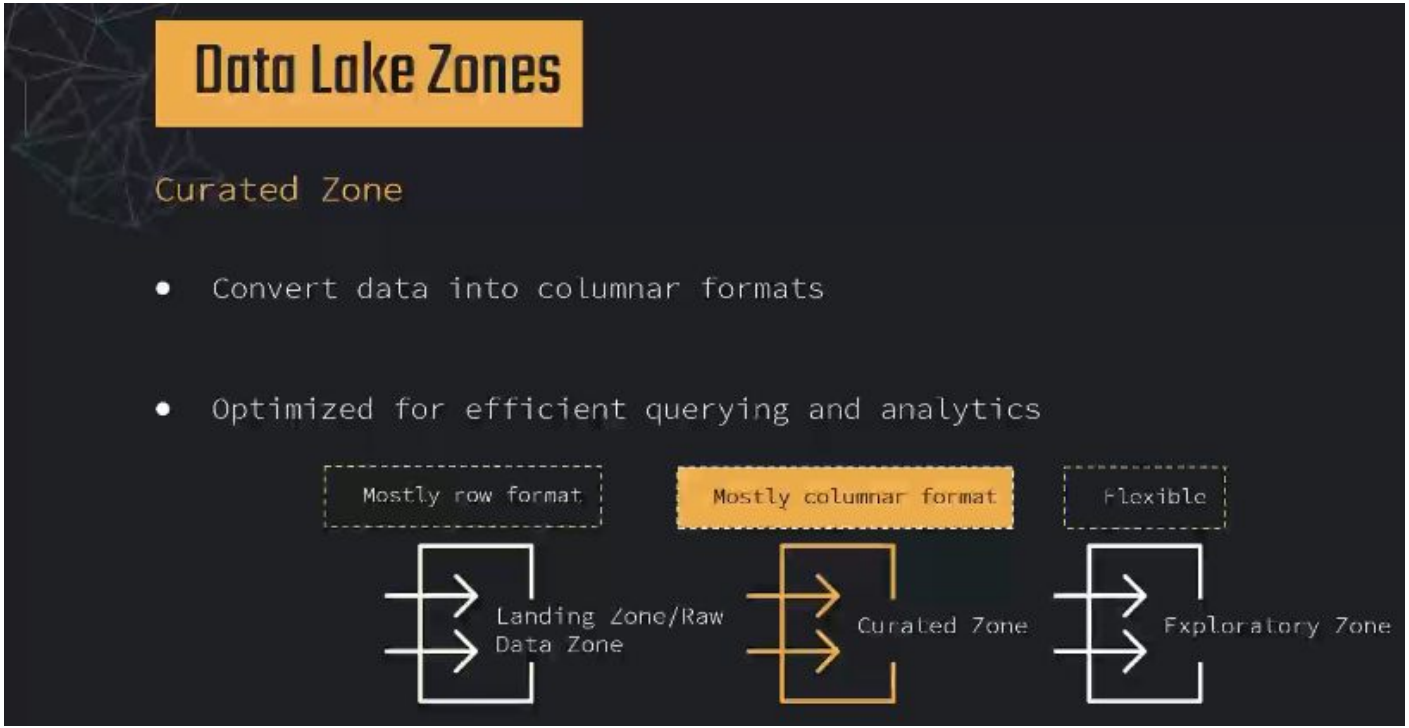
Data Formats used in a Data Lake



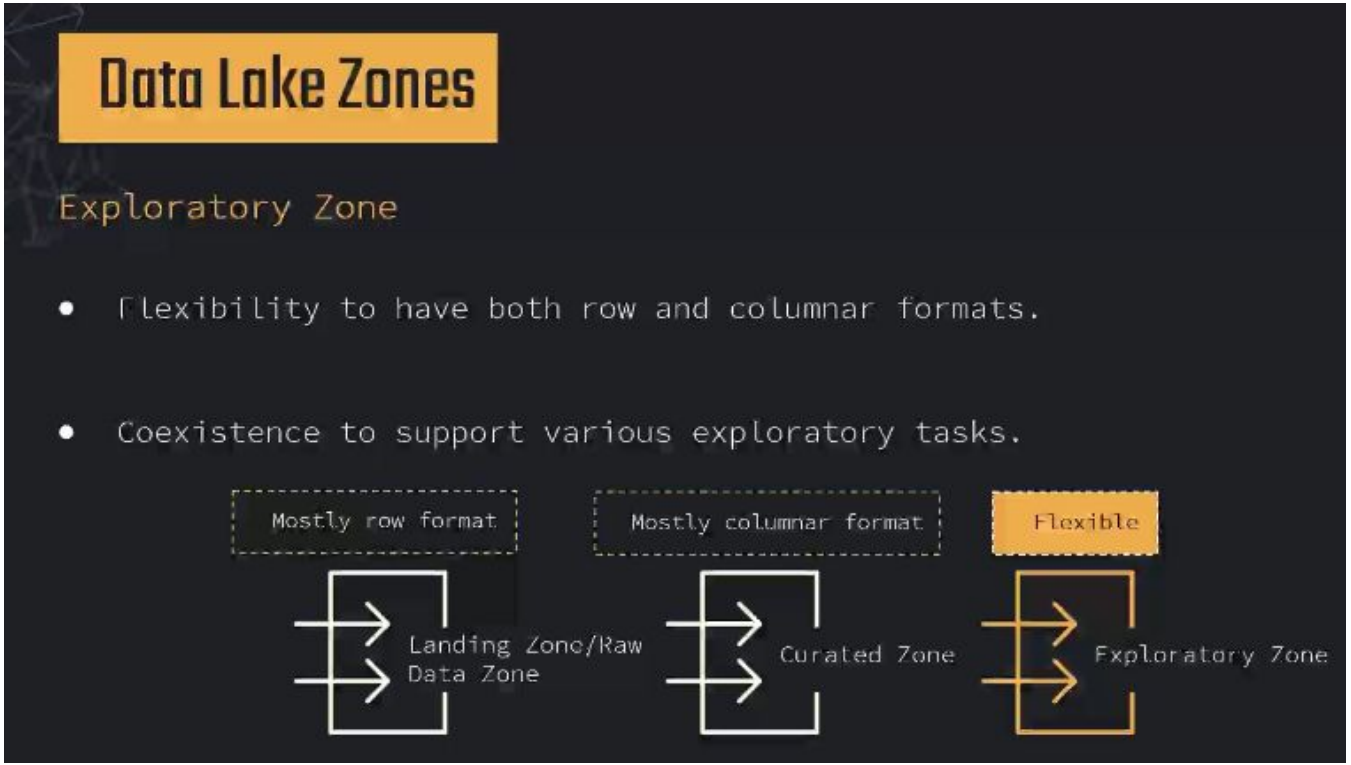
Data Formats used in a Data Lake



Data Formats used in a Data Lake



Data Formats used in a Data Lake



Amazon Athena & Apache Hive

- Cả hai đều là **SQL engine** chạy trên dữ liệu trong **data lake** (thường lưu ở HDFS hoặc Amazon S3).
- Không cần load dữ liệu vào RDBMS – dữ liệu được truy vấn trực tiếp ở chỗ nó nằm (schema-on-read).
- Dùng SQL quen thuộc: SELECT, JOIN, GROUP BY, Window function…
- Hỗ trợ các định dạng file phổ biến: CSV, JSON, Parquet, ORC, Avro.

Amazon Athena & Apache Hive

Tiêu chí	Amazon Athena	Apache Hive
Hạ tầng	Serverless, chạy trên AWS, không cần quản lý cluster	Chạy trên Hadoop cluster (on-premise hoặc cloud), cần quản lý hạ tầng
Engine bên dưới	Dựa trên **Presto/Trino** (tối ưu truy vấn phân tán in-memory)	Truyền thống chạy trên **MapReduce**, sau nâng cấp sang Tez hoặc Spark
Hiệu năng	Thường nhanh hơn vì Presto tối ưu cho interactive queries	Thường chậm hơn (MapReduce batch-oriented), Spark/Tez thì khá hơn
Chi phí	Trả tiền theo **dữ liệu quét (scan)**, ví dụ $5 / TB	Tốn chi phí vận hành cluster (máy chủ, nhân sự)
Quản lý schema	Dùng Glue Data Catalog hoặc Hive Metastore	Dùng Hive Metastore
Mục tiêu chính	Ad-hoc analytics, BI, nhanh gọn trên S3	Batch ETL, Data Warehouse query trên Hadoop

Distributed Databases

Định nghĩa: CSDL phân tán, dữ liệu lưu trên nhiều node/địa điểm (scale ngang / dọc)

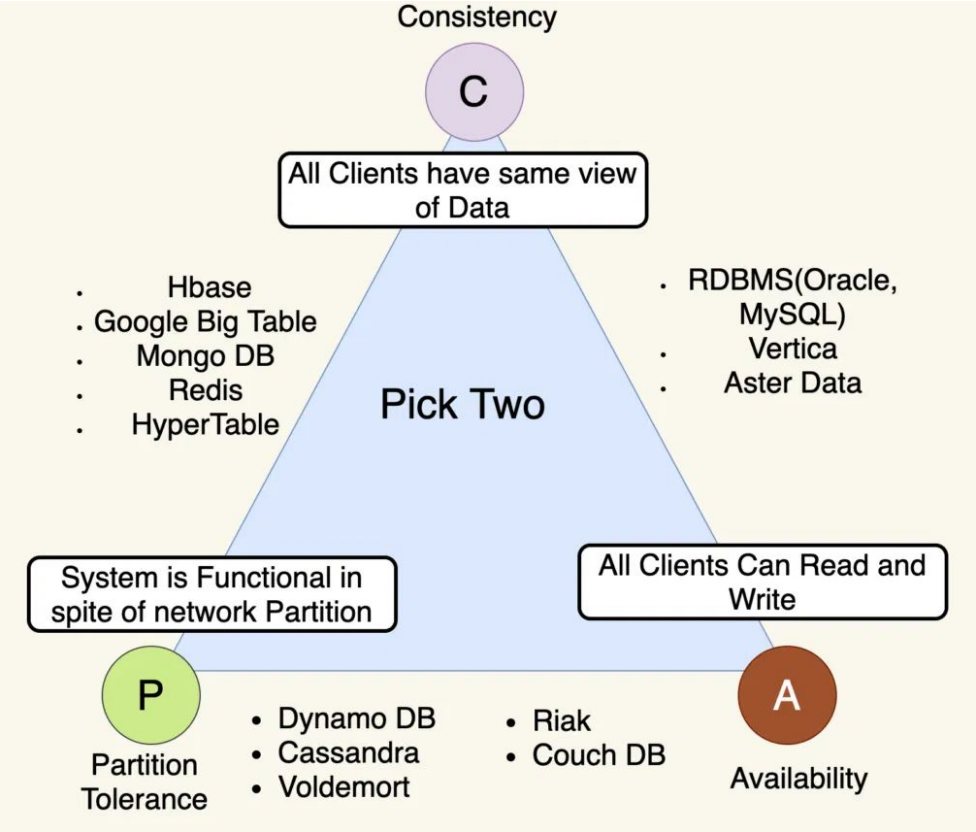
Đặc điểm chính

- Transparency: user thấy như 1 DB duy nhất
- Data replication & partitioning
- Distributed query processing

Thách thức

- CAP theorem

CAP Theorem


Consistency
C
All Clients have same view of Data
Hbase
Google Big Table
Mongo DB
Redis
HyperTable
RDBMS(Oracle, MySQL)
Vertica
Aster Data
Pick Two
System is Functional in spite of network Partition
All Clients Can Read and Write
P
Partition Tolerance
Dynamo DB
Cassandra
Voldemort
Riak
Couch DB
A
Availability

CAP Theorem - CA (Consistency + Availability)

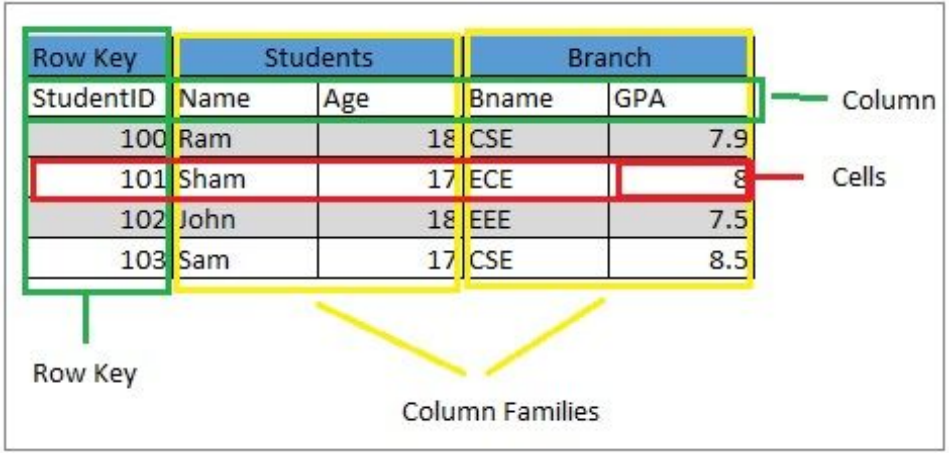
- Giả định **không có partition** (mạng luôn ổn định).
- Mỗi request nhận được phản hồi **ngay lập tức** và dữ liệu **luôn nhất quán**.
- Thực tế: hầu như không tồn tại trong hệ phân tán lớn (vì partition là không tránh khỏi).
- Ví dụ gần nhất: **Single-node RDBMS** (Oracle, SQL Server chạy trên 1 server).
    - Mặc định: MySQL, PostgreSQL, SQL Server, Oracle chủ yếu scale dọc.
    - Đọc mở rộng (read scaling): dùng replication.
    - Viết mở rộng (write scaling): cần sharding hoặc cluster, phức tạp hơn nhiều so với distributed database gốc (Cassandra, Spanner…).

CAP Theorem - CP (Consistency + Partition tolerance)

- Khi có partition, hệ thống chọn **tính nhất quán** hơn là sẵn sàng.
- Có thể từ chối hoặc delay request để đảm bảo dữ liệu đúng.
- Phù hợp cho hệ thống **tài chính, ngân hàng** (ưu tiên tính chính xác).
- Ví dụ: **HBase, MongoDB (chế độ strict consistency)**

CAP Theorem - CP - HBase

- **Data model**: dạng bảng nhưng **NoSQL**
- **Row key**: định danh duy nhất cho dòng
- **Column Family**
  - Nhóm các cột liên quan
  - Dữ liệu trong 1 family được lưu chung → tối ưu truy vấn
- **Schema-on-read**: linh hoạt, không cần schema cứng
  - Không cần định nghĩa trước tất cả cột.
  - Chỉ cần định nghĩa Column Family
  - Trong mỗi family, có thể thêm cột mới bất kỳ lúc nào (column qualifier), khác nhau giữa từng row.
  - Mỗi cell (xác định bởi RowKey + ColumnFamily + ColumnQualifier) có thể có nhiều version.



CAP Theorem - CP - HBase

Kiểu phân tán: Master–Slave

HMaster

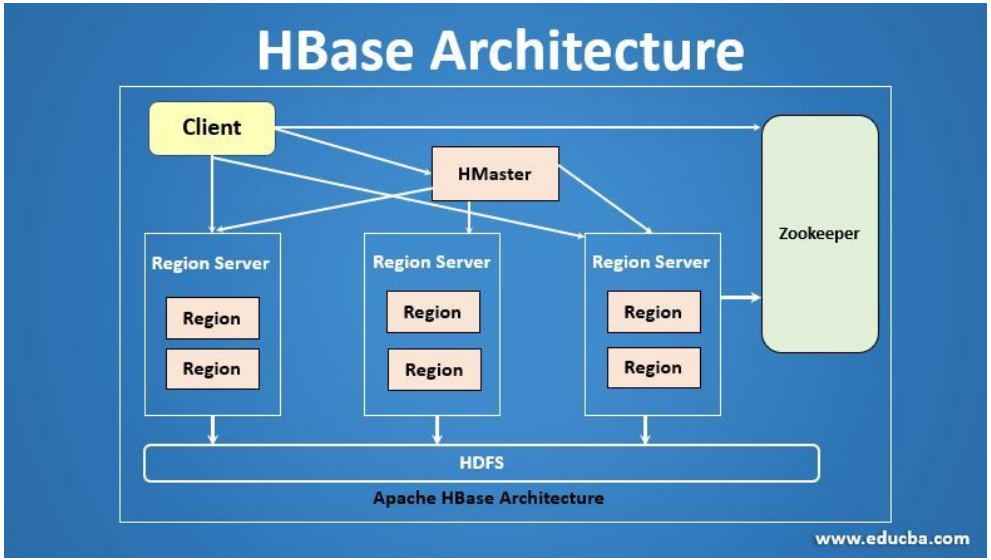
- Quản lý metadata, phân vùng (region assignment)
- Theo dõi trạng thái RegionServer

RegionServer (Slave)

- Lưu trữ dữ liệu theo region (một dải row key)
- Xử lý đọc/ghi từ client

ZooKeeper

- Điều phối cluster, phát hiện lỗi node



CAP Theorem - CP - HBase

Vì sao HBase là CP (Consistency + Partition Tolerance)

- **Consistency (C)**
  - Mọi ghi (write) vào một row key luôn đi đến **RegionServer duy nhất** quản lý region đó → đảm bảo không có ghi song song gây lệch dữ liệu.
  - **Write-Ahead Log (WAL)**: trước khi ghi dữ liệu vào MemStore/HFile, HBase ghi log trước → đảm bảo dữ liệu không mất và nhất quán sau crash.
  - Client chỉ đọc/ghi từ RegionServer đang "chính chủ" quản lý region → kết quả luôn nhất quán.
- **Partition Tolerance (P)**
  - Khi có sự cố mạng (partition), cluster vẫn tiếp tục hoạt động:
    - ZooKeeper phát hiện RegionServer nào mất kết nối → HMaster gán lại region cho node khác.
    - WAL giúp replay dữ liệu, đảm bảo không mất update.
- **Đổi lại Availability (A) bị giảm**
  - Nếu RegionServer hoặc kết nối đến nó bị partition, client có thể **tạm không truy cập được dữ liệu của region đó**, nhưng toàn cluster không sập.
  - HBase chọn "thà từ chối hoặc chờ request còn hơn trả dữ liệu sai".

CAP Theorem - AP (Availability + Partition tolerance)

- Khi có partition, hệ thống vẫn **trả kết quả nhanh**, nhưng dữ liệu có thể **không nhất quán tạm thời** (eventual consistency).
- Thích hợp cho hệ thống **social media, e-commerce, caching** (ưu tiên uptime).
- Ví dụ: **Cassandra, DynamoDB, CouchDB**.

CAP Theorem - AP - Cassandra

Data model: **Wide-column store** (giống HBase)

- Dữ liệu theo **Keyspace** → **Column Family** → **Row** → **Column**

Replication & Fault tolerance

- **Peer-to-Peer architecture** (không có master).
- Tự động **replicate** dữ liệu nhiều node / data center.

NoSQL đặc trưng

- High availability, không có single point of failure.
- Eventual consistency (nếu chọn mức thấp).

